LINUX FEST 2026 · WORKSHOP

Kubernetes Fundamentals

From container orchestration to running real apps on a cluster



Saman Hoseini

What you'll leave with



Why orchestration

The problems Kubernetes solves and the desired-state idea behind it



How a cluster works

Control plane, nodes, and what really happens on kubectl apply



Ship an app

Deploy, configure, expose, and roll out new versions safely



Run it well

Right-size resources, autoscale, secure, and debug workloads

Plan for the day

Welcome	Goals and plan
Part 1 · Concepts	Orchestration, architecture, workloads, config & storage, networking
Break & cluster setup	Coffee, then install minikube and check your cluster
Part 2 · Hands-on	Deploy, break and heal, configure, expose, debug, challenge round
Short break	
Part 3 · Advanced	Custom endpoints, Gateway API, NetworkPolicy, resources, autoscaling, RBAC, security, challenge round, static Pods
Wrap-up & Q&A	Next steps and questions

PART 1

Concepts

Orchestration, architecture, workloads, config & storage, networking. No laptop needed: just follow along.



01

Container Orchestration

From one container on a laptop to hundreds across a fleet

Quick recap: what a container is



Image

Immutable, layered, versioned package of your app and its dependencies



Namespaces

Isolation: own view of processes, network, mounts, hostname, users



cgroups

Limits: how much CPU, memory and I/O the process may use

Great on one machine.

But production is many machines, many containers, and things that fail at 3 AM.

Who decides where each container runs, how many, and what happens when one dies?

Life without orchestration

Your app: 3 × API, Postgres, Redis, spread across 5 servers. Now answer:



Placement

Which server has room for the next container?



Failure

Who restarts a crashed container, or moves it off a dead server?



Scale

Traffic doubles at 9 PM. Who adds replicas, and removes them later?



Discovery

Containers move and change IPs. How do they find each other?



Releases

How do we ship v2 with zero downtime, and roll back fast?



Config

Where do configs and passwords live, and who can read them?

Container orchestration

> *Container orchestration automatically provisions, deploys, scales, and manages containerized applications without worrying about the underlying infrastructure.*

Google Cloud · cloud.google.com/discover/what-is-container-orchestration

What an orchestrator gives you



Scheduling

Places containers on nodes with enough resources



Self-healing

Restarts, replaces and reschedules failed containers



Scaling

Adds or removes replicas, manually or automatically



Discovery & LB

Stable names and load balancing for moving targets



Rollouts

Rolling updates with health checks and rollbacks



Config & secrets

Injects configuration without rebuilding images



Resource mgmt

CPU and memory requests, limits and quotas



Storage

Attaches the right volume wherever a container lands

Docker Compose vs an orchestrator

	Docker Compose	Kubernetes
Scope	One host	A cluster of many nodes
Best for	Local dev, demos, small setups	Production, high availability
Scaling	Manual: --scale on one host	Manual or automatic (HPA), across nodes
Self-healing	Restart policy on the same host	Reschedules onto healthy nodes
Networking	Single-host bridge + DNS	Cluster-wide Services, Ingress, policies
Rollouts	Stop and recreate	Rolling updates, health-gated, rollback
Learning curve	Low	Higher (that is why we are here)

A short history of orchestration

2003
Borg
Google's internal cluster manager

2013
Docker
Containers go mainstream

2014
Kubernetes
Open-sourced by Google, inspired by Borg

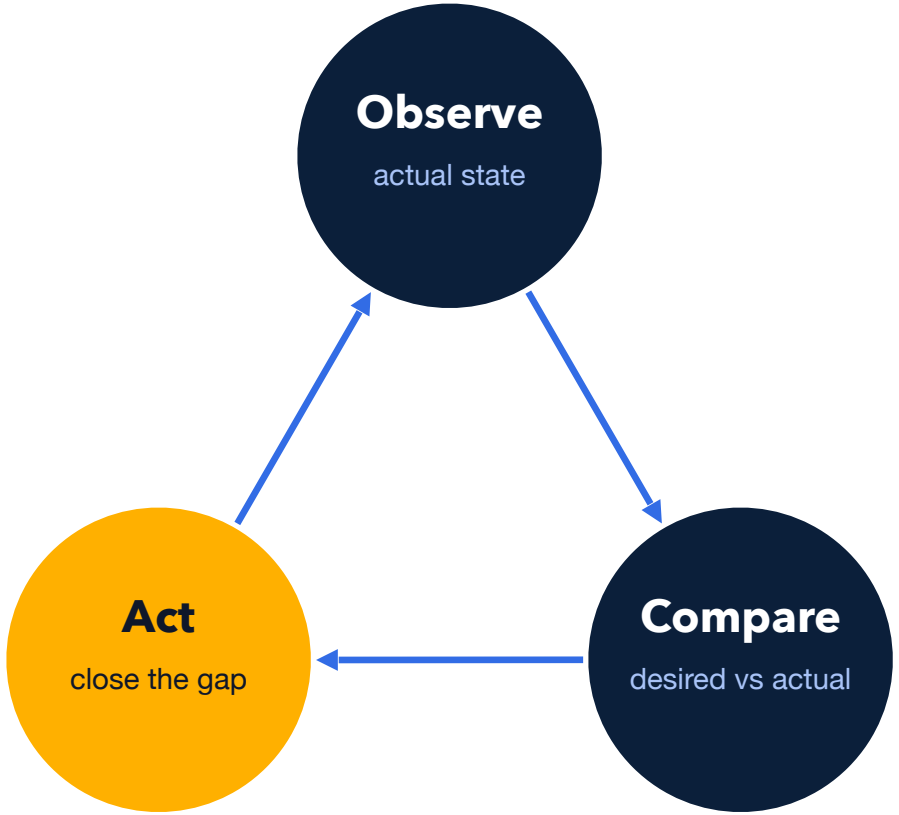
2015
v1.0 + CNCF
Kubernetes donated to the new CNCF

2016–17
The orchestrator wars
Swarm, Mesos/ Marathon, Nomad, Kubernetes

2018
CNCF graduation
Kubernetes is the first graduated project

2022
v1.24
dockershim removed; CRI runtimes standard

The core idea: desired state



Declarative, not imperative

You describe what you want. Controllers keep reality matching it, forever.

```
spec:
  replicas: 3          # desired
status:
  readyReplicas: 2     # actual -> +1 Pod
```

- **Idempotent:** apply the same file 10 times, same result
- **Self-healing:** drift is corrected automatically
- **Git-friendly:** the YAML is the source of truth (GitOps)

Why Kubernetes won

Declarative API

Everything is a resource in YAML, reconciled continuously

Extensible

Custom resources + operators teach it new tricks

Runs anywhere

Every cloud, bare metal, edge, your laptop

Vendor-neutral

CNCF-governed, huge contributor base

Ecosystem

Helm, Argo CD, Prometheus, Istio, cert-manager…

Batteries included

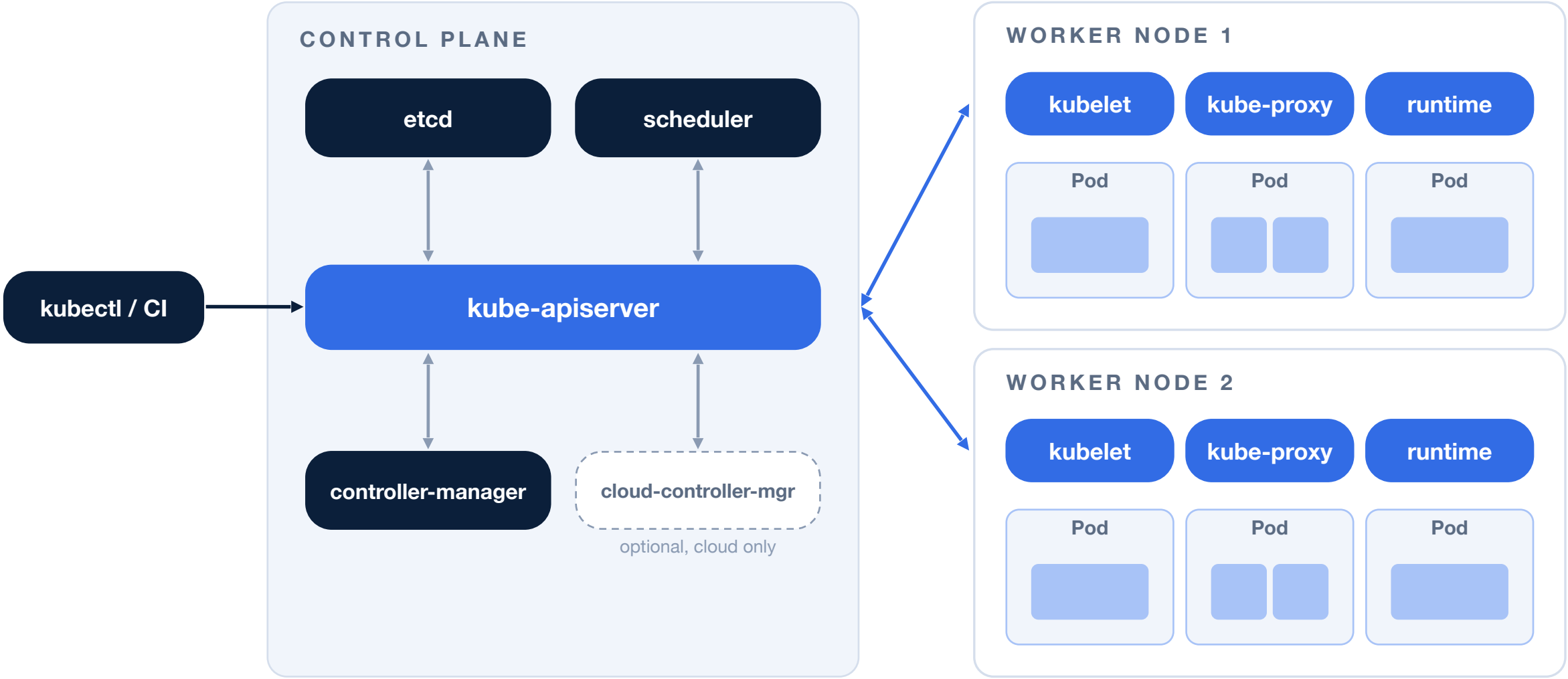
Rollouts, scaling, RBAC, quotas, policies built in

02

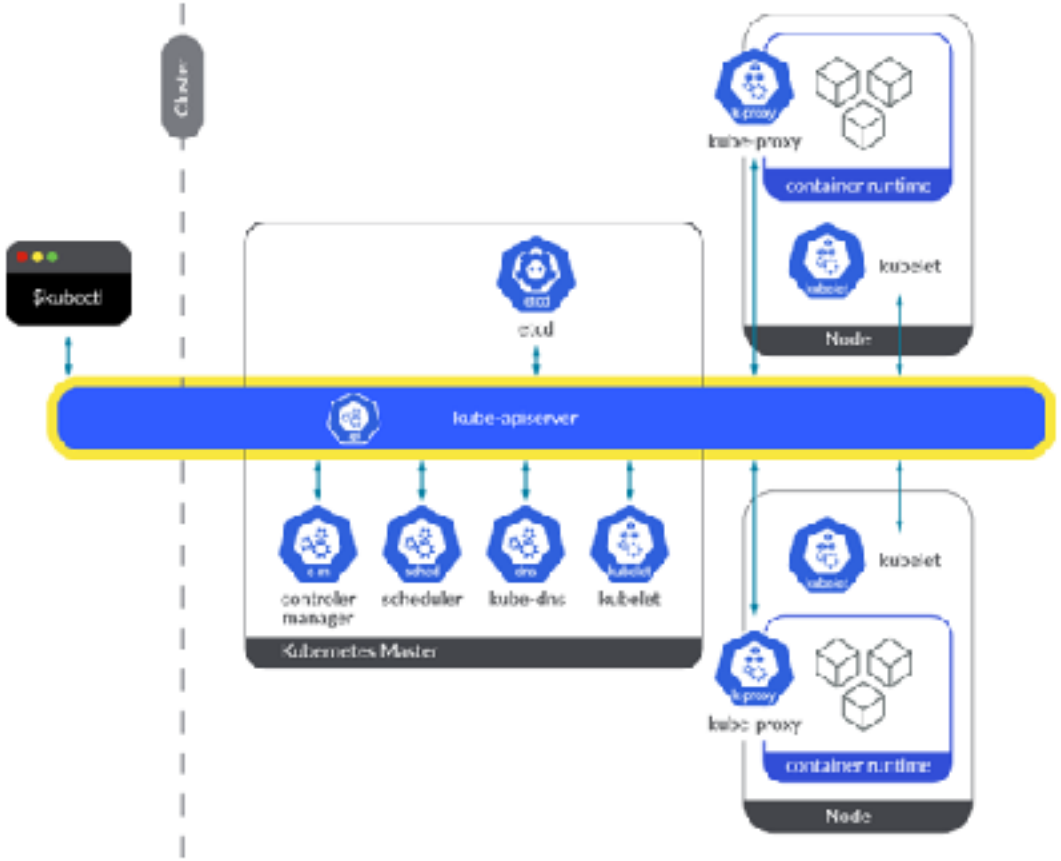
Kubernetes Architecture

The control plane decides. The nodes do the work.

A cluster at a glance



kube-apiserver: the front door



- RESTful API: the only way into the cluster
- Authenticates, authorizes (RBAC) and runs admission checks
- The only component that reads and writes etcd
- Clients watch it for changes instead of polling
- Stateless: run several replicas for high availability

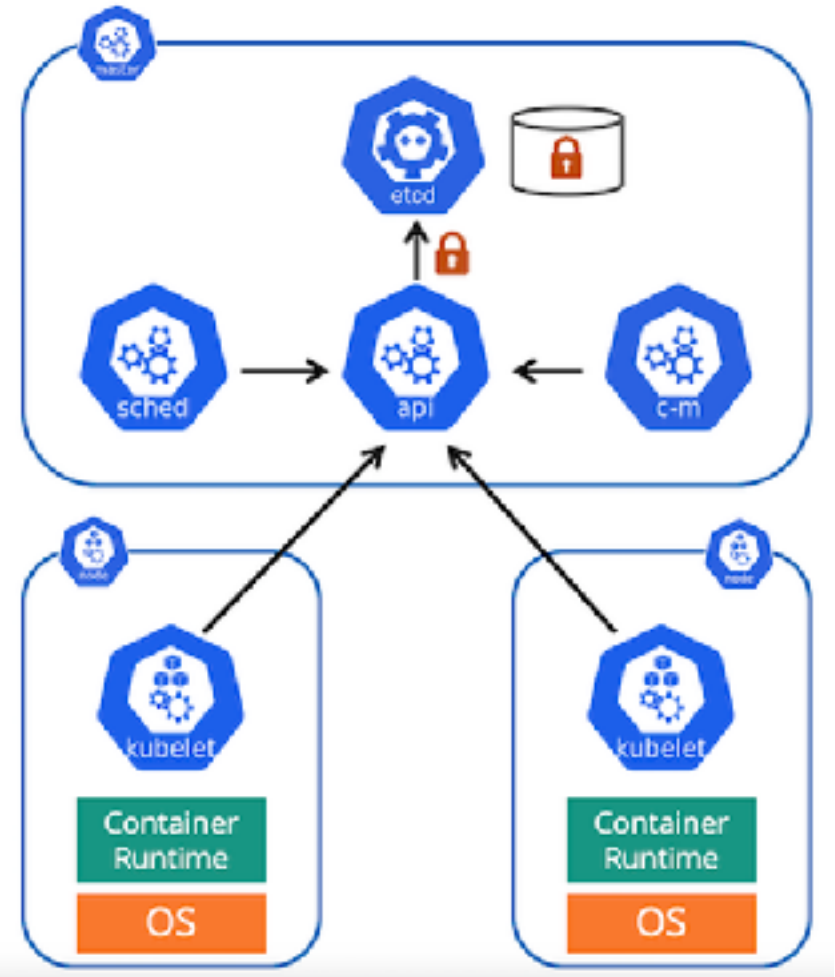
```
request -> authn -> authz -> admission -> etcd
```

Life of an API request

kubectl · CI · controllers · kubelets



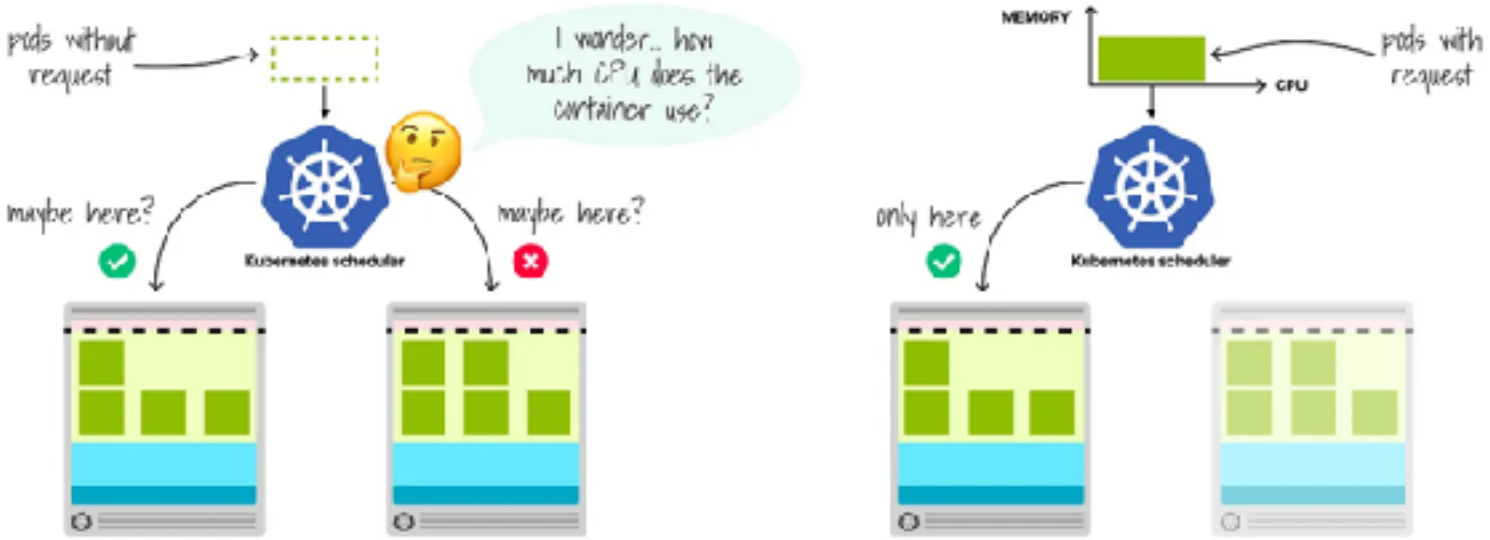
etcd: the cluster's memory



- Consistent, distributed key-value store (Raft)
- Holds all cluster state: objects, config, Secrets
- Only the API server talks to it
- Run 3 or 5 members: a majority must agree
- Back it up. No backup = no cluster after a disaster

```
# admin side: snapshot etcd
$ ETCDCTL_API=3 etcdctl snapshot save /backup/etcd.db \
    --endpoints=https://127.0.0.1:2379 \
    --cacert=ca.crt --cert=server.crt --key=server.key
```

kube-scheduler: where should this Pod run?



1 Filter

Drop nodes that can't fit the Pod: requests, taints, selectors, ports

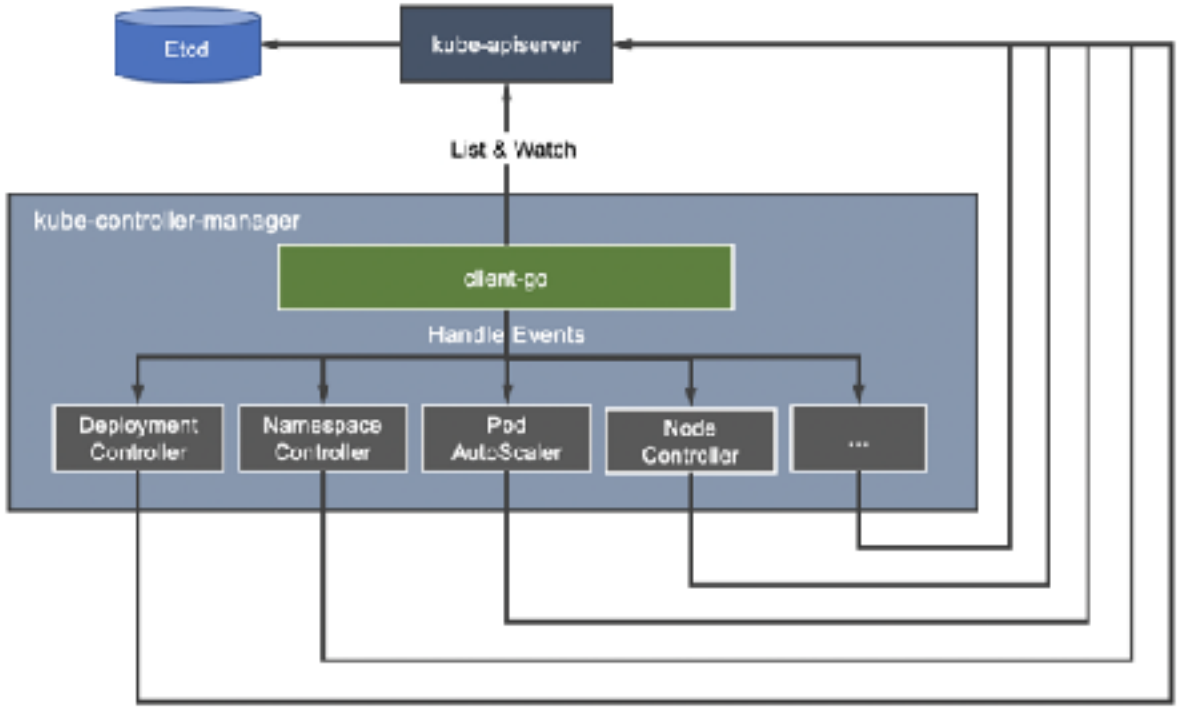
2 Score

Rank the rest: spreading, affinity, balanced usage, image locality

3 Bind

Write the chosen node to the Pod. kubelet takes it from there

kube-controller-manager: the reconcilers



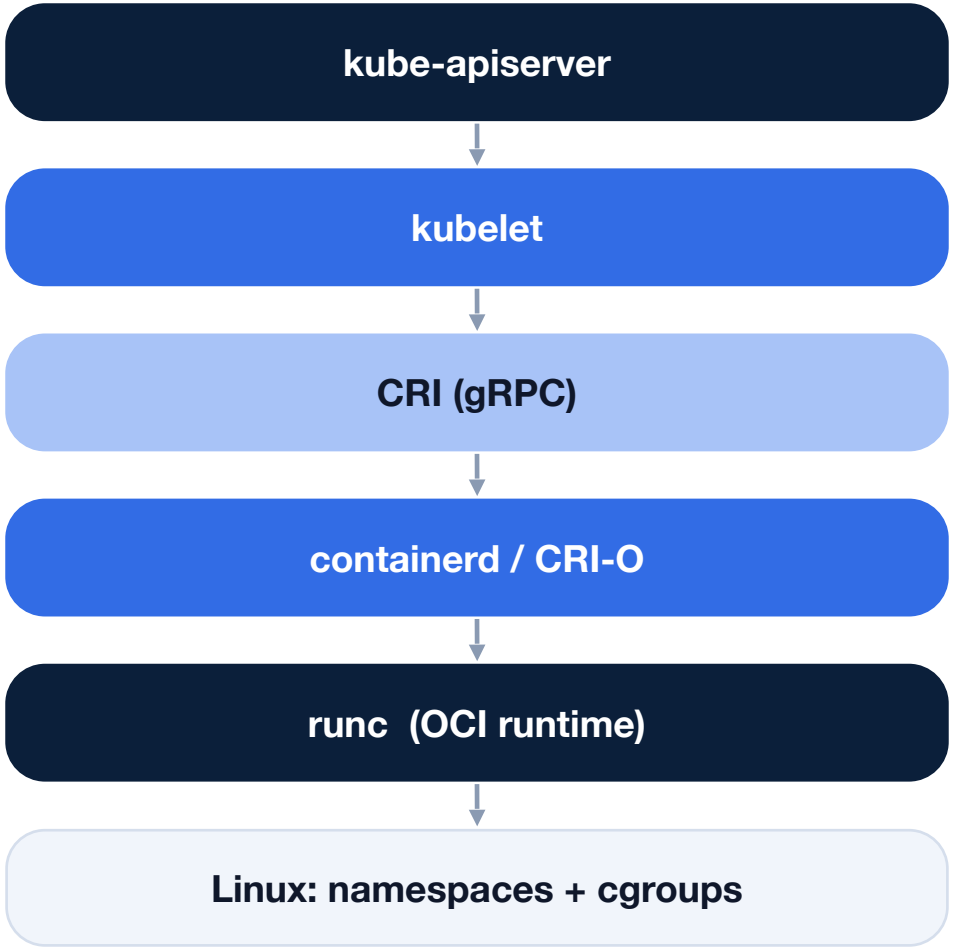
Deployment / ReplicaSet	keep the right number of Pods at the right version
Node	notices unhealthy nodes and evicts their Pods
Job / CronJob	run Pods to completion, on a schedule
Endpoints	track which Pod IPs back each Service
ServiceAccount	create default identities per namespace

Each controller = one observe → compare → act loop

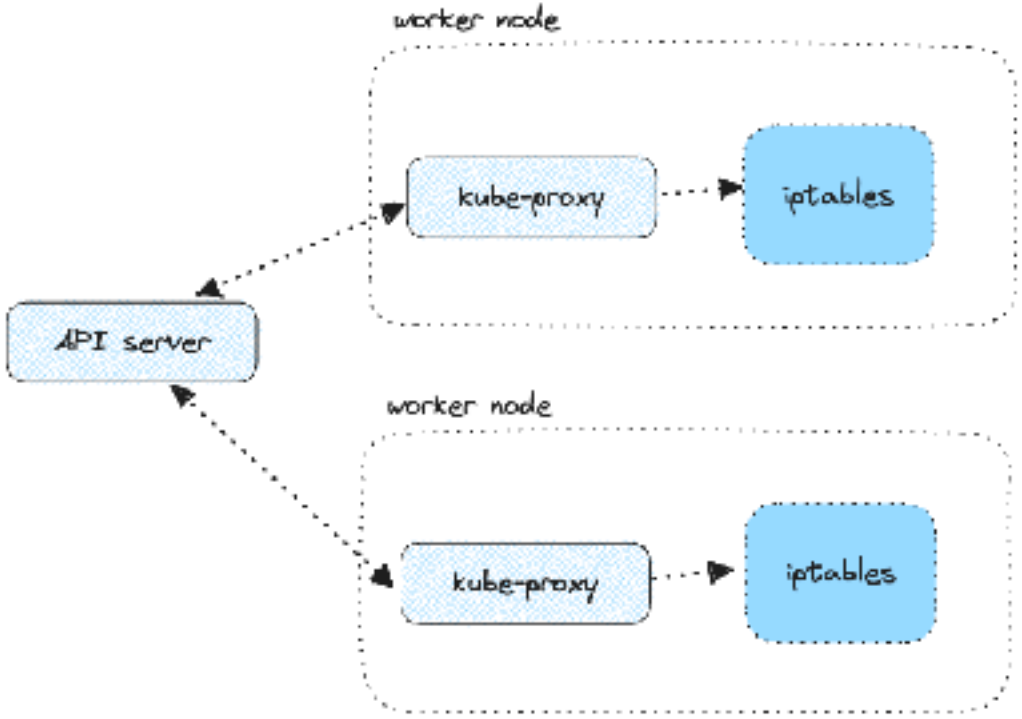
kubelet: the node agent

- Runs on every node; registers it with the cluster
- Watches for Pods scheduled to its node
- Asks the container runtime (via CRI) to start them
- Runs liveness, readiness and startup probes
- Mounts volumes, injects config and Secrets
- Reports Pod and node status back to the API server

Fun fact: kubeadm runs the control plane as static Pods from /etc/kubernetes/manifests



kube-proxy: making Services work



- Runs on every node (usually as a DaemonSet)
- Watches Services and their endpoints via the API server
- Programs iptables / IPVS / nftables rules
- Service virtual IP → one of the healthy Pod IPs
- Some CNIs (e.g. Cilium) replace it with eBPF

A Service IP is not a machine. It's a forwarding rule on every node.

Container runtimes and the CRI

High-level runtime (CRI)

- **containerd**
- **CRI-O**

Pull images, manage containers, talk to kubelet

Low-level runtime (OCI)

- **runc**
- **crun**

Create namespaces + cgroups, start the process

Sandboxed runtimes

- **gVisor**
- **Kata Containers**

Extra isolation via user-space kernel or micro-VM

Myth: “Kubernetes dropped Docker, so my images break.” Only the dockershim adapter was removed (v1.24). Images from docker build are OCI images and run unchanged.

Add-ons every cluster needs



CNI plugin

Pod networking + NetworkPolicy: Calico, Cilium, Flannel



CoreDNS

Cluster DNS: my-svc.my-ns.svc.cluster.local



Ingress / Gateway controller

HTTP routing into the cluster: Envoy Gateway, Traefik, NGINX Gateway Fabric…



CSI driver

Provisions and attaches persistent volumes



metrics-server

CPU / memory metrics for kubectl top and autoscaling



Cloud controller

Cloud load balancers, routes, node lifecycle

Who runs the control plane?



Managed

- EKS, GKE, AKS, DigitalOcean, and local cloud providers
- Provider runs control plane + etcd
- You own workloads, often nodes
- Fastest path to production



Self-managed

- kubeadm, k3s, RKE2, Talos, Kubespray
- You run etcd, certs, upgrades
- Full control, on-prem / air-gapped
- Needs a platform team



Local / dev

- minikube, k3d, OrbStack
- Disposable clusters in seconds
- Perfect for learning and CI
- Not for production traffic

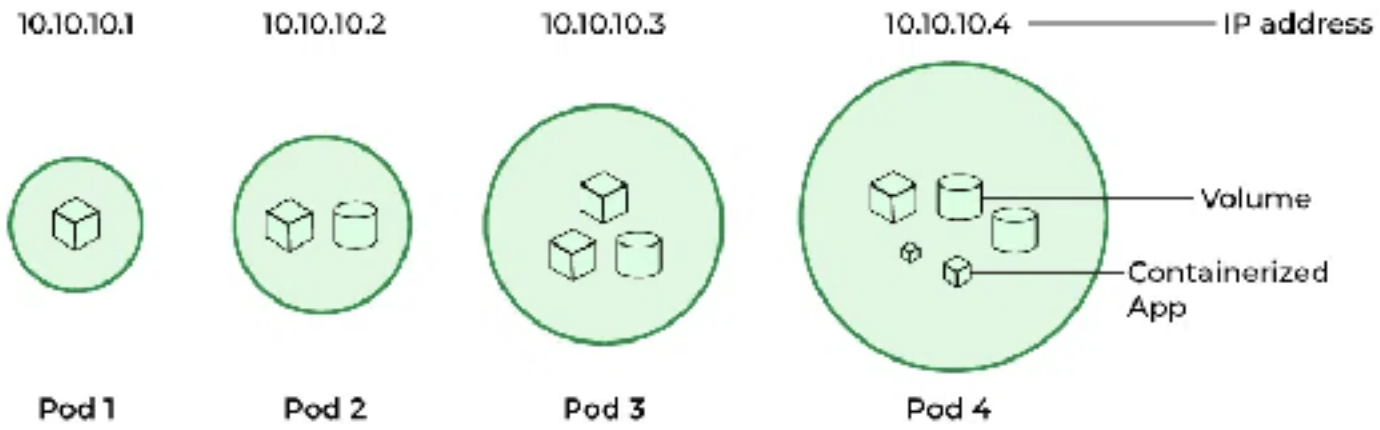
03

Core Workloads

Pods, labels, ReplicaSets, Deployments, rollouts and health probes



Pod: the smallest deployable unit



Shared network

One IP per Pod. Containers talk over localhost

Shared volumes

Containers can mount the same volumes

Same node

Always scheduled together as a unit

Ephemeral

Replaced, not repaired. New Pod = new IP

Anatomy of a Kubernetes manifest

web.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: shop
  labels:
    app: web
spec:
  replicas: 3
  # ... what you want
status:
  readyReplicas: 3
  # ... what you have (read-only)
```

apiVersion + kind

Which API group/version and object type

metadata

Name, namespace, labels, annotations

spec

Desired state: you write this

status

Actual state: Kubernetes writes this

```
$ kubectl explain deployment.spec.strategy
```

Multi-container Pods: init & sidecar

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels: { app: web }
spec:
  initContainers:
  - name: migrate              # runs once, first
    image: ghcr.io/acme/web:1.4
    command: ["./migrate", "up"]
  - name: log-shipper          # native sidecar
    image: fluent/fluent-bit:3.2
    restartPolicy: Always
  containers:
  - name: app
    image: ghcr.io/acme/web:1.4
    ports:
    - containerPort: 8080
```

Init container

Runs to completion before the app. Migrations, wait-for-dependency, fetch config.

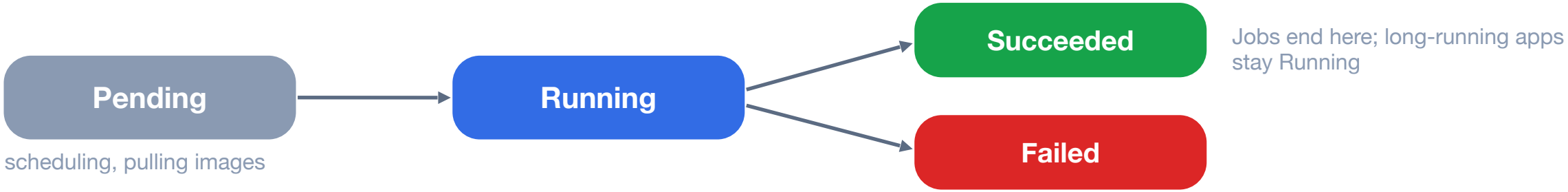
Sidecar

Runs next to the app for its whole life. Log shipping, proxies, cert reloaders.

Rule of thumb

Separate services go in separate Pods. Same Pod only if they must share a lifecycle.

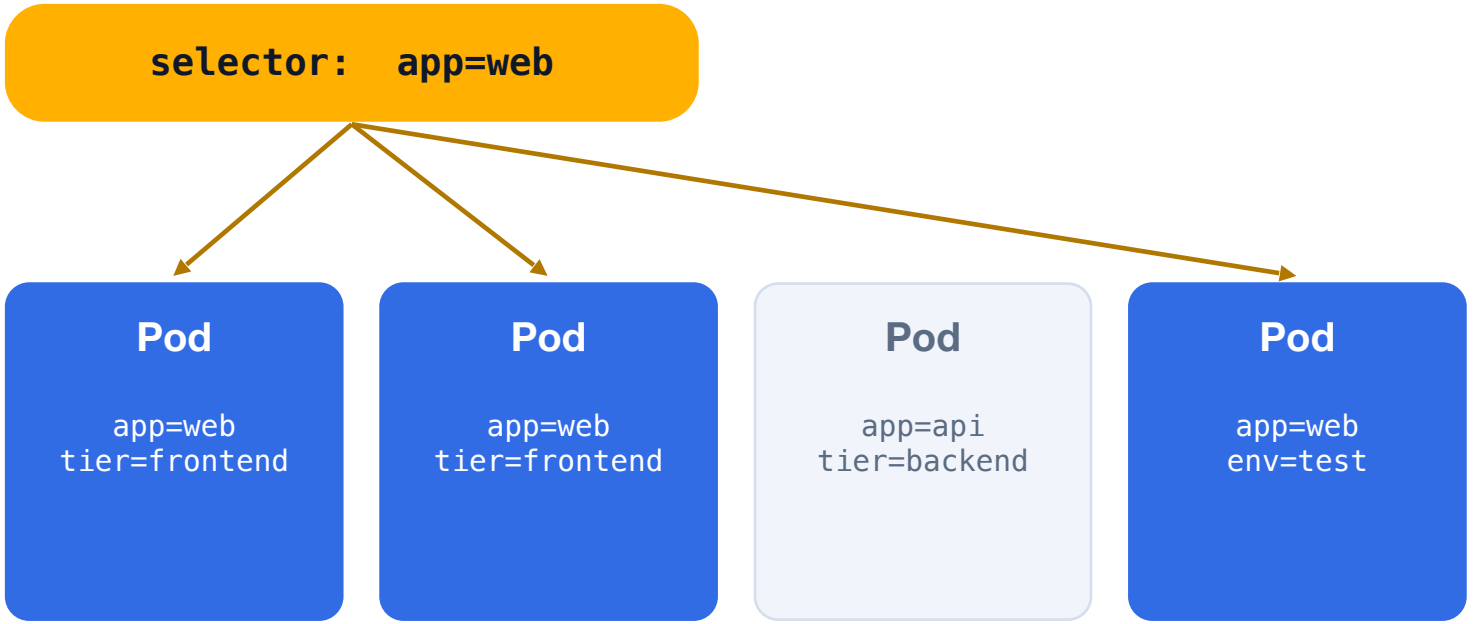
Pod lifecycle and the statuses you'll see



Statuses you'll actually see

`ContainerCreating`	Pulling image, mounting volumes. Should pass quickly
`ImagePullBackOff`	Wrong image name/tag, or missing registry credentials
`CrashLoopBackOff`	Container keeps exiting. Check logs --previous
`OOMKilled`	Exceeded its memory limit and was killed by the kernel
`Evicted`	Node ran low on memory/disk; Pod was removed

Labels & selectors: the glue



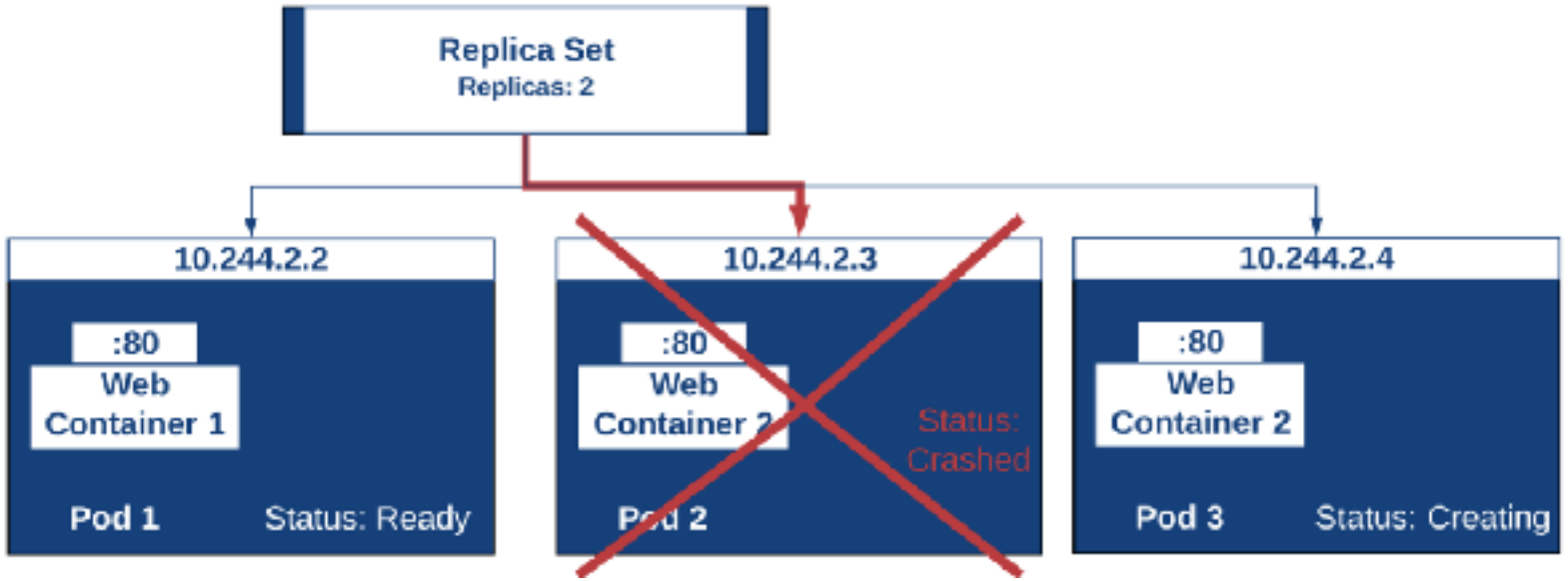
3 of 4 Pods match → those are the Service's endpoints

```
# select by label
$ kubectl get pods -l app=web
$ kubectl get pods -l 'env!=prod'

# add / change a label
$ kubectl label pod web-7d9 env=test

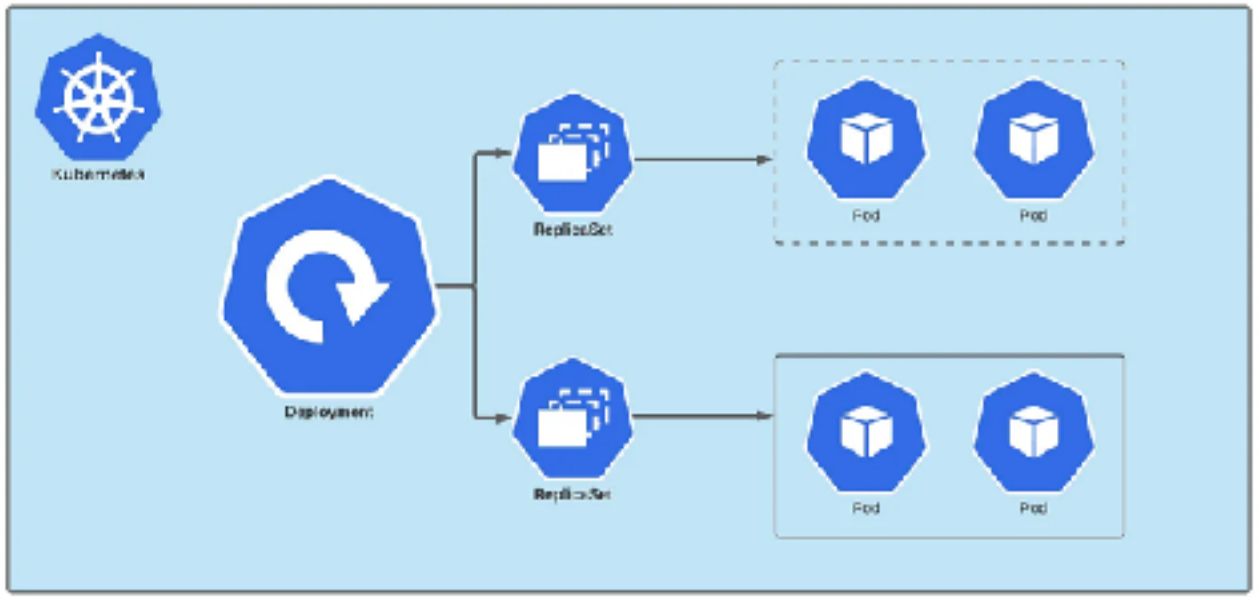
# show labels as columns
$ kubectl get pods -L app,tier
```

ReplicaSet: keep N copies alive



- Ensures the desired number of Pod replicas are running
- Replaces Pods that crash, get deleted, or are lost with a node
- Counts Pods via its label selector
- Managed by a Deployment: you rarely touch it directly

Deployment: versioned, rolling releases



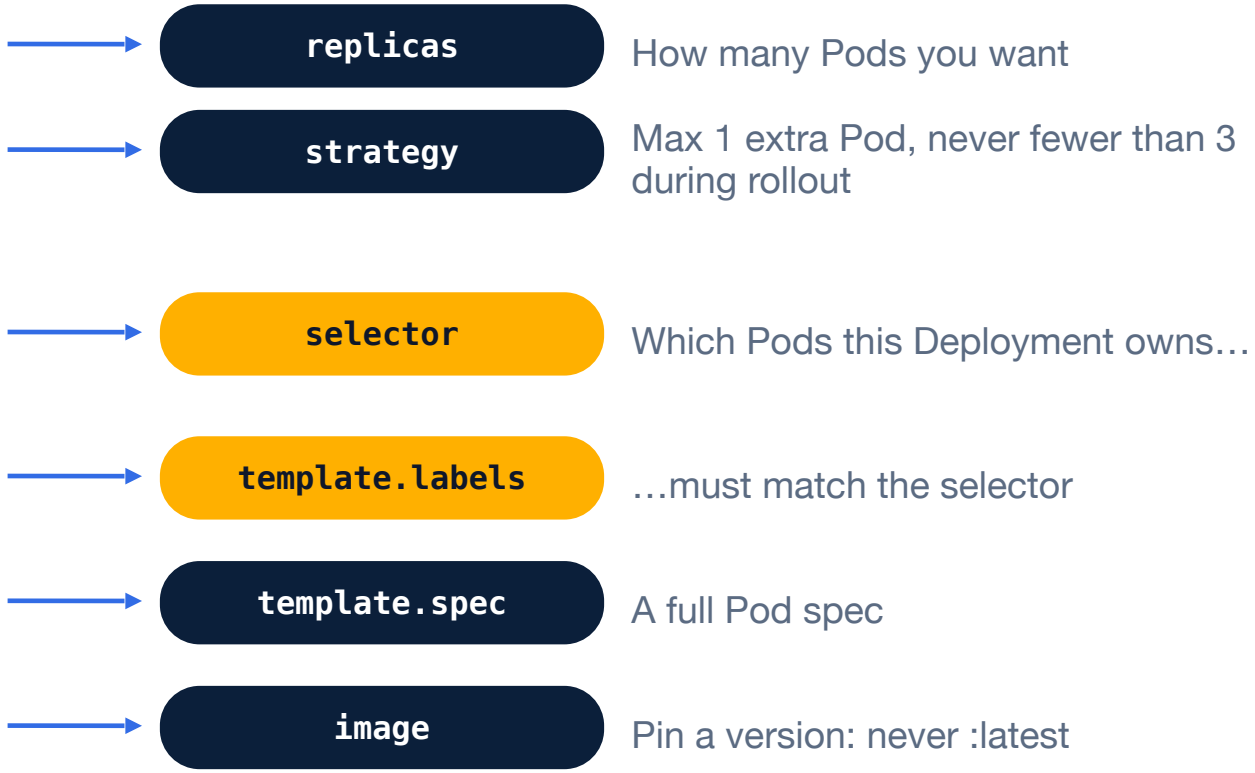
- Owns ReplicaSets; each template change = new ReplicaSet
- Rolling updates with zero downtime
- Revision history + one-command rollback
- Scale up/down by changing replicas
- The default choice for stateless apps

```
Deployment web
 └─ ReplicaSet web-7d9f  (v1.4, 0 replicas)   # kept for rollback
 └─ ReplicaSet web-5c8b  (v1.5, 3 replicas)
      ├─ Pod web-5c8b-x2k
      ├─ Pod web-5c8b-p9q
      └─ Pod web-5c8b-t7m
```

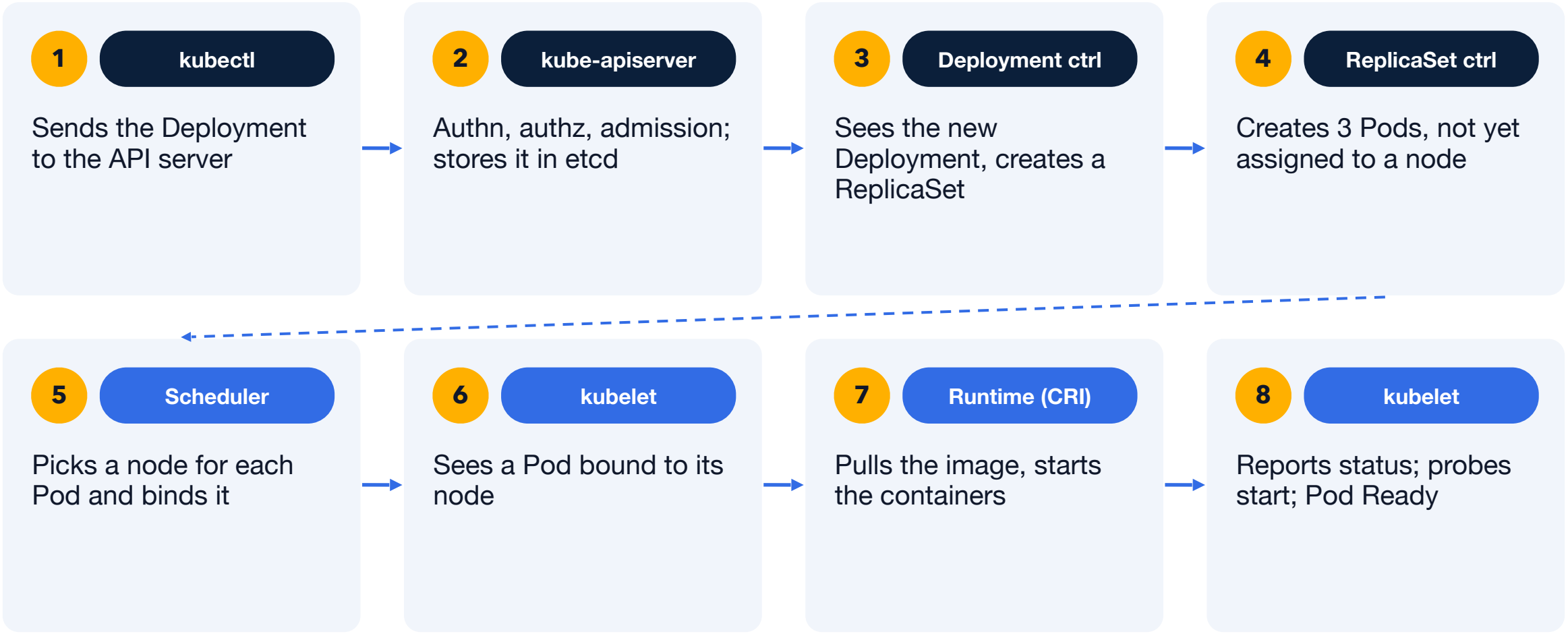
A Deployment, field by field

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:1.27
        ports:
        - containerPort: 80
```



What happens on kubectl apply?



Rolling updates & rollbacks



maxSurge: 1 · maxUnavailable: 0 → capacity never drops

```
# ship a new version
$ kubectl set image deploy/web web=nginx:1.28
$ kubectl rollout status deploy/web

# inspect history
$ kubectl rollout history deploy/web

# something's wrong? go back
$ kubectl rollout undo deploy/web

# restart Pods (e.g. after a Secret change)
$ kubectl rollout restart deploy/web
```

Health probes: tell Kubernetes how you feel



`startupProbe`

“Have I finished booting?”

Holds off the other probes until it passes. For slow starters (JVM, big caches).

On failure: restart after failureThreshold



`readinessProbe`

“Can I take traffic right now?”

Not ready → removed from Service endpoints. Gates rolling updates.

On failure: no restart, just no traffic



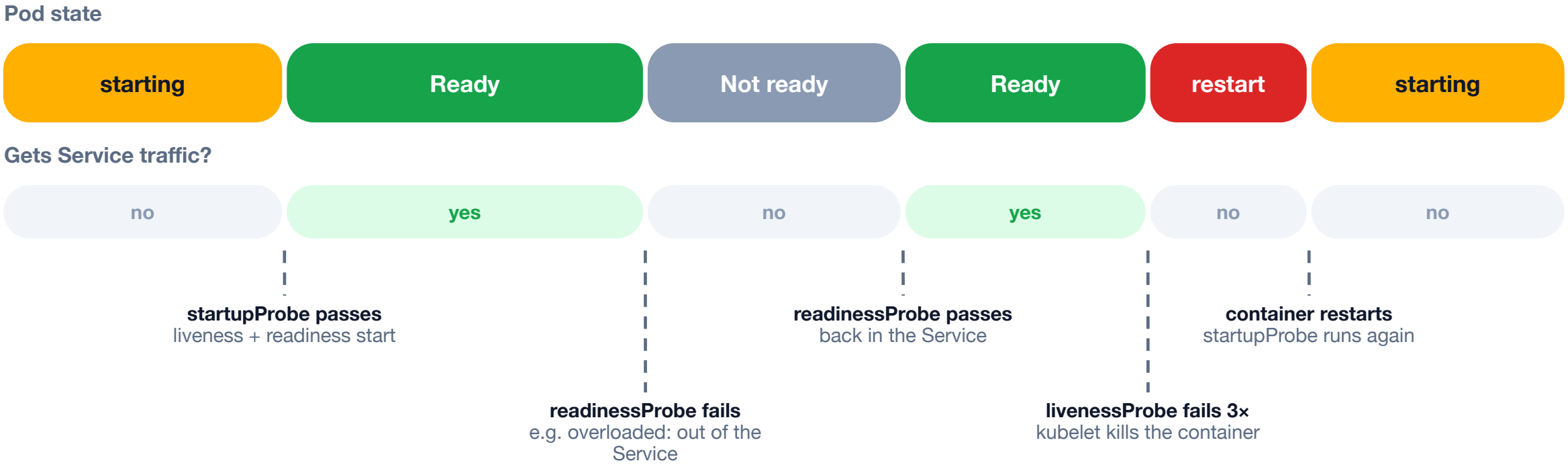
`livenessProbe`

“Am I stuck beyond repair?”

Detects deadlocks and hangs that won't fix themselves.

On failure: container is restarted

Probes over a Pod's lifetime



`startupProbe fails`

Container restarted after failureThreshold

`readinessProbe fails`

No traffic, no restart. Recovers by itself

`livenessProbe fails`

Container restarted. Only for stuck processes

Probes in practice

```yaml
containers:
- name: api
  image: ghcr.io/acme/api:2.3.1
  ports:
  - containerPort: 8080
  startupProbe:
    httpGet: { path: /livez, port: 8080 }
    failureThreshold: 30   # 30 x 2s = 60s max
    periodSeconds: 2
  readinessProbe:
    httpGet: { path: /readyz, port: 8080 }
    periodSeconds: 5
  livenessProbe:
    httpGet: { path: /livez, port: 8080 }
    periodSeconds: 10
    failureThreshold: 3
```

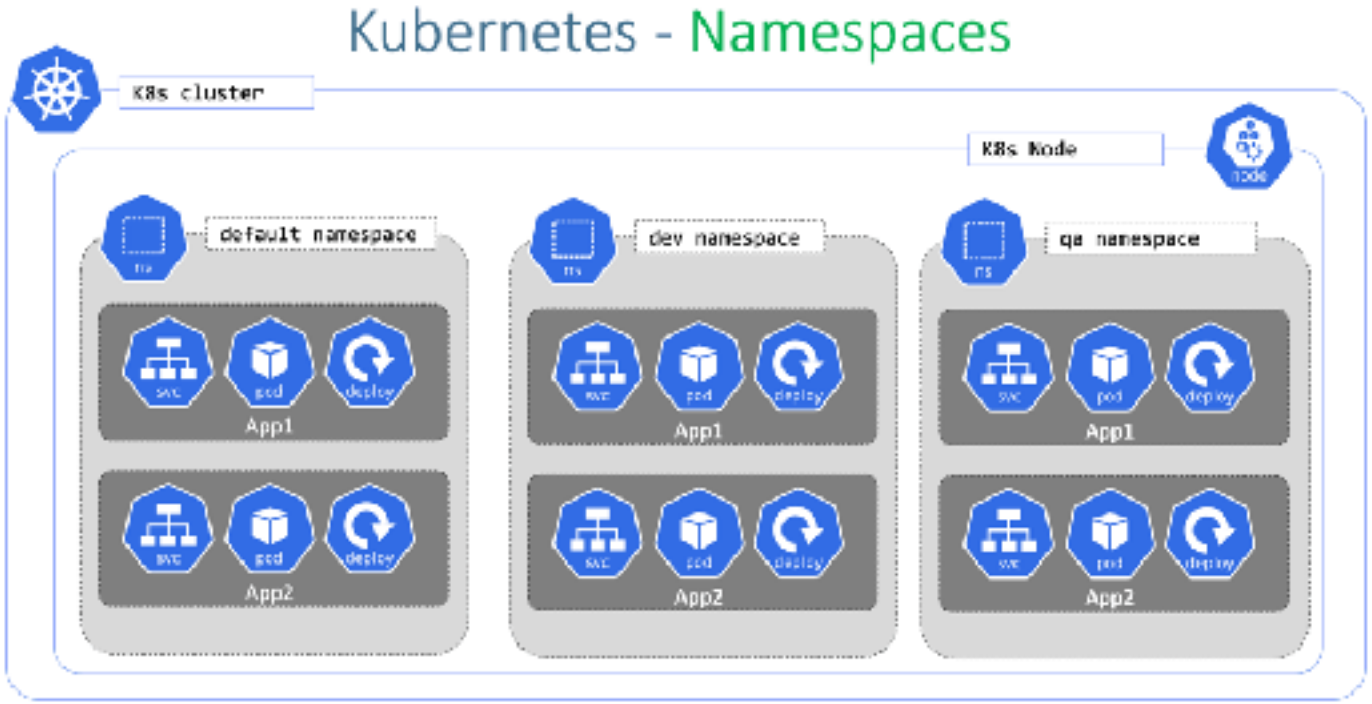
Do

- Keep probe endpoints fast and cheap
- Readiness may check what this instance needs to serve
- Use startupProbe instead of a huge initialDelaySeconds

Don't

- Check the database in liveness: one blip restarts every replica
- Point liveness and readiness at a slow, heavy endpoint
- Skip readiness: rollouts send traffic to booting Pods

Namespaces: virtual clusters



- Scope for names, RBAC, quotas and policies
- Split by team, app or environment
- Not a network boundary by default
- Some objects are cluster-wide: Nodes, PVs, StorageClasses

```
$ kubectl get ns
$ kubectl create namespace shop
$ kubectl get pods -n kube-system
$ kubectl config set-context --current --namespace=shop
```

04

Config & Storage

ConfigMaps, Secrets, volumes, PersistentVolumes, and stateful workloads

ConfigMap & Secret

ConfigMap

Non-sensitive key/value config: URLs, flags, whole config files. Change config without rebuilding the image.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-config
data:
  LOG_LEVEL: "info"
  FEATURE_X: "true"
```

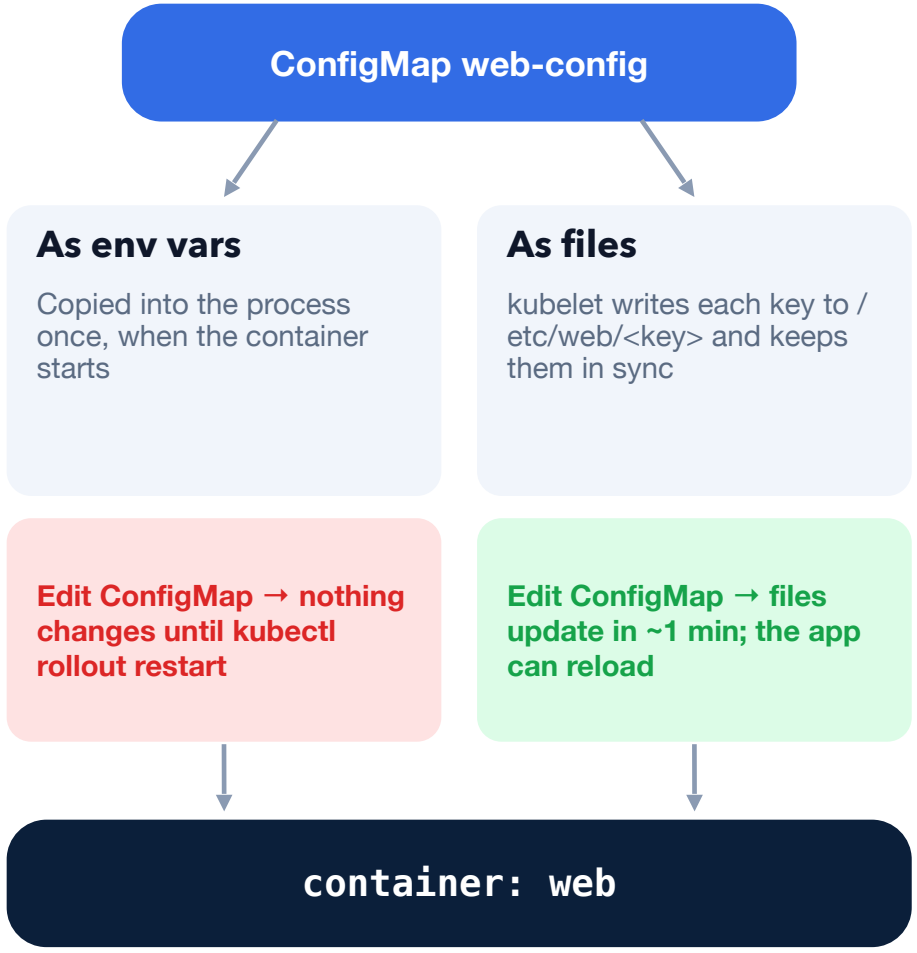
Secret

Passwords, tokens, keys, TLS certs. **base64 is encoding, not encryption.** Lock down with RBAC + encryption at rest.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
stringData:      # stored as base64
  DB_PASSWORD: "s3cr3t"
```

Using config in a Pod

```yaml
spec:
  containers:
  - name: web
    image: ghcr.io/acme/web:1.5
    envFrom:                          # all keys as env vars
    - configMapRef: { name: web-config }
    env:
    - name: DB_PASSWORD               # a single key
      valueFrom:
        secretKeyRef: { name: db-creds, key: DB_PASSWORD }
    volumeMounts:
    - name: config
      mountPath: /etc/web             # files: /etc/web/<key>
      readOnly: true
  volumes:
  - name: config
    configMap: { name: web-config }
```



Secrets work exactly the same way

Volumes: ephemeral vs persistent



Ephemeral

Lives and dies with the Pod

`emptyDir`	scratch space, share files between containers (medium: Memory = tmpfs)
`configMap / secret`	config as files
`projected`	several sources in one directory, incl. ServiceAccount tokens

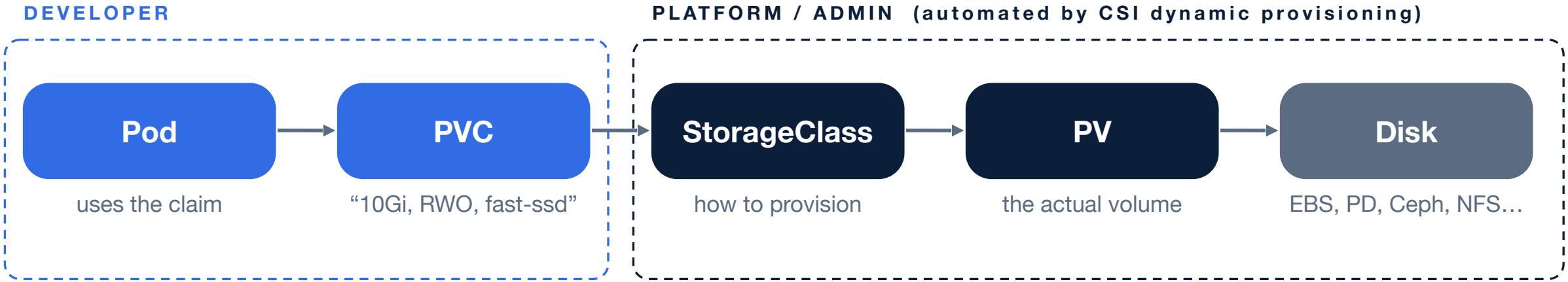


Persistent

Outlives Pods and rescheduling

`persistentVolumeClaim`	the normal way: ask for storage, get a PV
`CSI drivers`	cloud disks, Ceph, NFS, Longhorn, local NVMe
`hostPath`	a node's directory: avoid outside system agents

PersistentVolumes, Claims & StorageClasses




```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 10Gi
```

```yaml
# in the Pod spec
volumes:
- name: data
  persistentVolumeClaim:
    claimName: data
```

Mode	Meaning
RWO	one node read-write
ROX	many nodes read-only
RWX	many nodes read-write
RWOP	a single Pod

Stateless vs stateful

Stateful apps need…

Stable identity

the same name and DNS record after a restart

Their own storage

each replica keeps its own volume

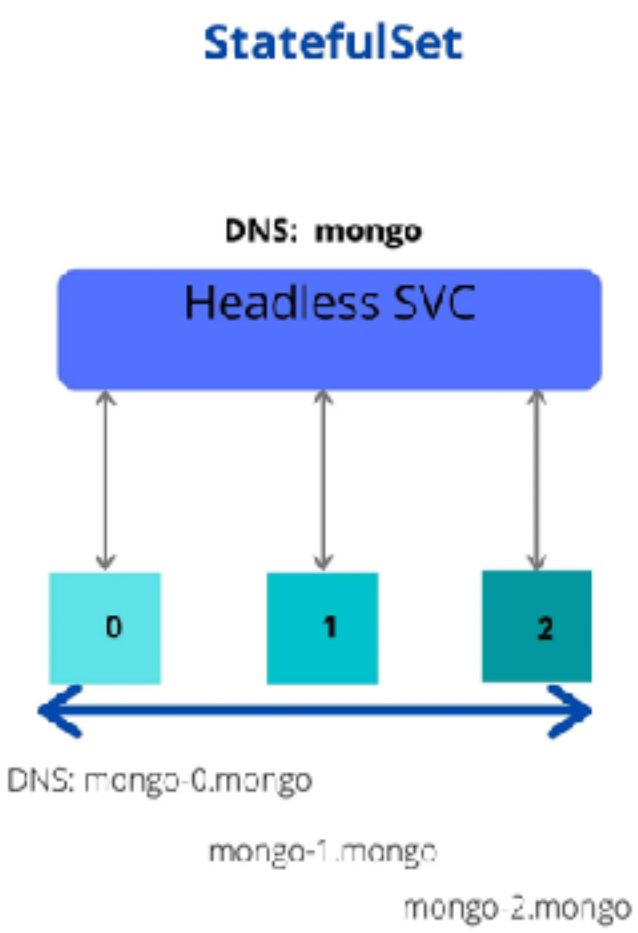
Order

start, stop and update one by one, in order

Your call: stateful or stateless?

1. A university website front end
2. MySQL
3. A WebSocket calculator
4. Kafka
5. A REST API with a database behind it

StatefulSet



- **Stable names:** db-0, db-1, db-2
- **Own storage:** volumeClaimTemplates → one PVC per Pod
- **Stable DNS:** via a headless Service (clusterIP: None)
- **Ordered:** create 0→N, delete N→0, update one at a time

```
# DNS for each Pod
db-0.db.shop.svc.cluster.local
```

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db  # headless
  replicas: 3
  selector:
    matchLabels: { app: db }
  template:
    metadata:
      labels: { app: db }
    spec:
      containers:
      - name: postgres
        image: postgres:17
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests: { storage: 10Gi }
```

Other workload types

DaemonSet

One Pod on every (or selected) node. New nodes get one automatically.

e.g. Log collectors, node exporters, CNI agents

```yaml
kind: DaemonSet
spec:
  template:
    spec:
      containers:
      - name: fluent-bit
```

Job

Runs Pods to completion, with retries. Can run in parallel.

e.g. DB migration, batch import, one-off backup

```yaml
kind: Job
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
```

CronJob

Creates a Job on a cron schedule.

e.g. Nightly backups, reports, cleanup tasks

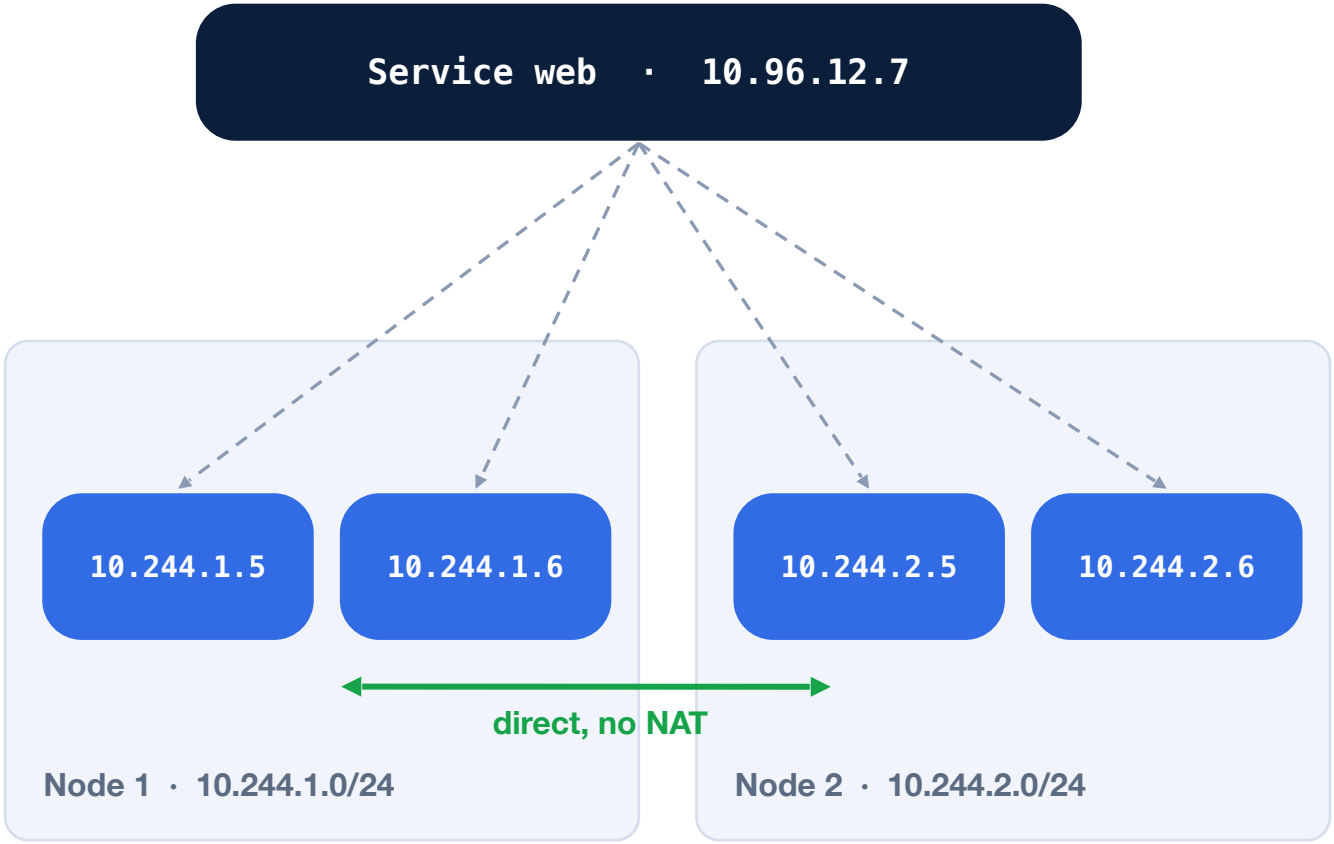
```yaml
kind: CronJob
spec:
  schedule: "0 2 * * *"
  timeZone: "Asia/Tehran"
  concurrencyPolicy: Forbid
```

05

Kubernetes Networking

Services, DNS and Ingress: how traffic reaches your Pods

The networking model



Rules of the road

- Every Pod gets its own cluster-wide IP
- Pods reach each other across nodes without NAT
- The CNI plugin makes it so
- Pod IPs change on every restart
- So: put a Service in front, with a stable IP + DNS name

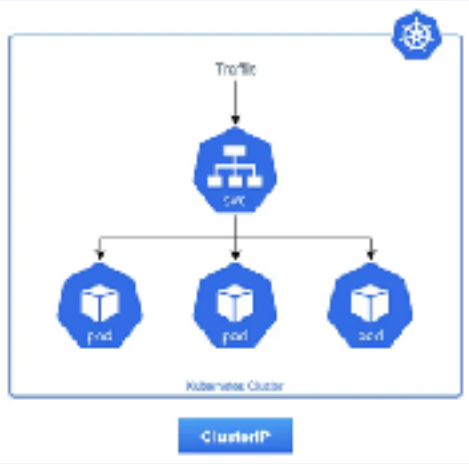
Service: a stable front for moving Pods

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP          # default
  selector:
    app: web               # matches Pod labels
  ports:
  - name: http
    port: 80               # what clients use
    targetPort: 8080       # container port
```

```
$ kubectl get endpoints web
```

- Stable virtual IP + DNS name
- Selects Pods by label
- Load-balances across ready Pods only
- port (Service) ≠ targetPort (container)
- No endpoints? Check the selector and readiness first

Service types



ClusterIP

Internal virtual IP. Default; use for service-to-service traffic



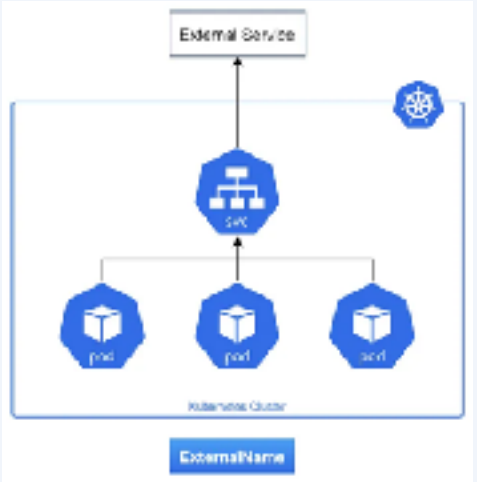
NodePort

Port 30000–32767 on every node. Labs and debugging



LoadBalancer

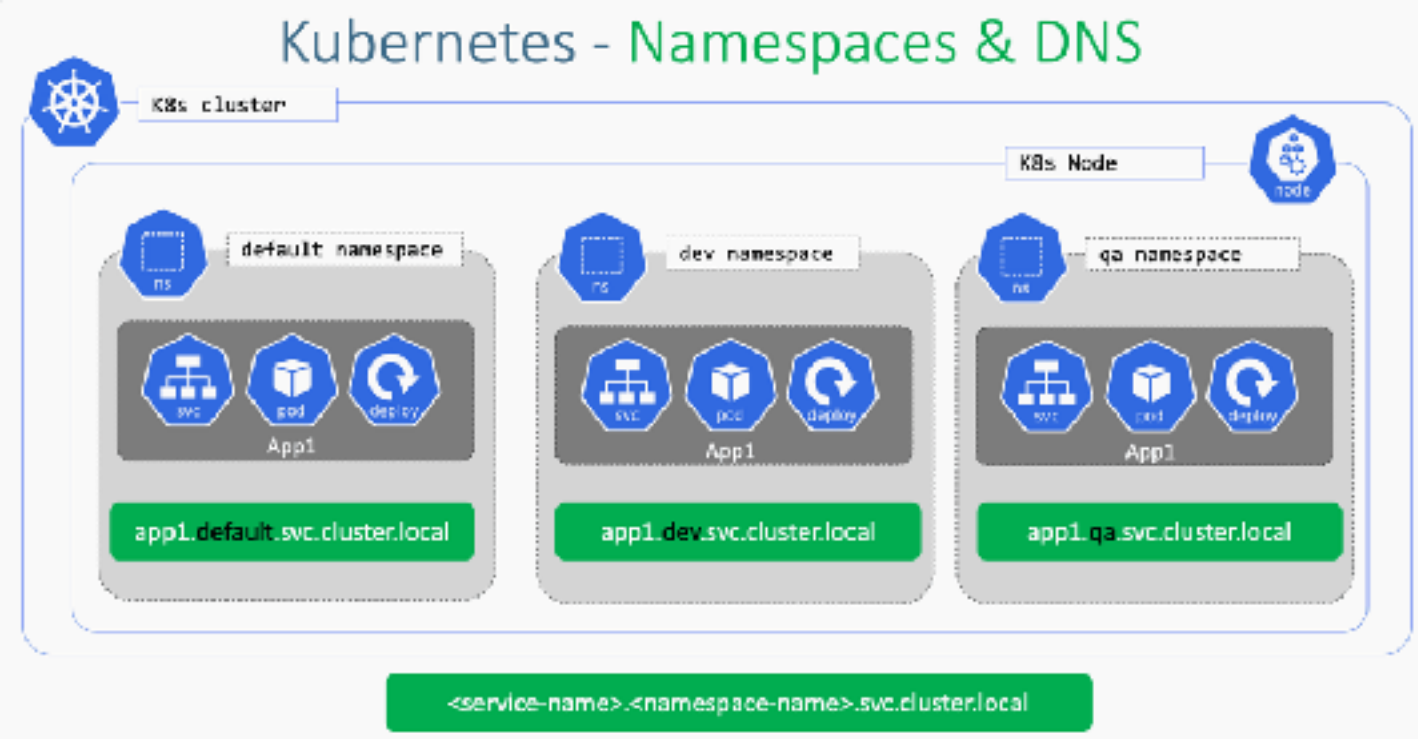
External LB from the cloud, or MetalLB / kube-vip on bare metal



ExternalName

DNS CNAME to an outside host. No proxying, no selector

Service discovery with DNS




```
<service>.<namespace>.svc.cluster.local
```

From a Pod in namespace shop:

```
web
```

→ web in shop

```
web.billing
```

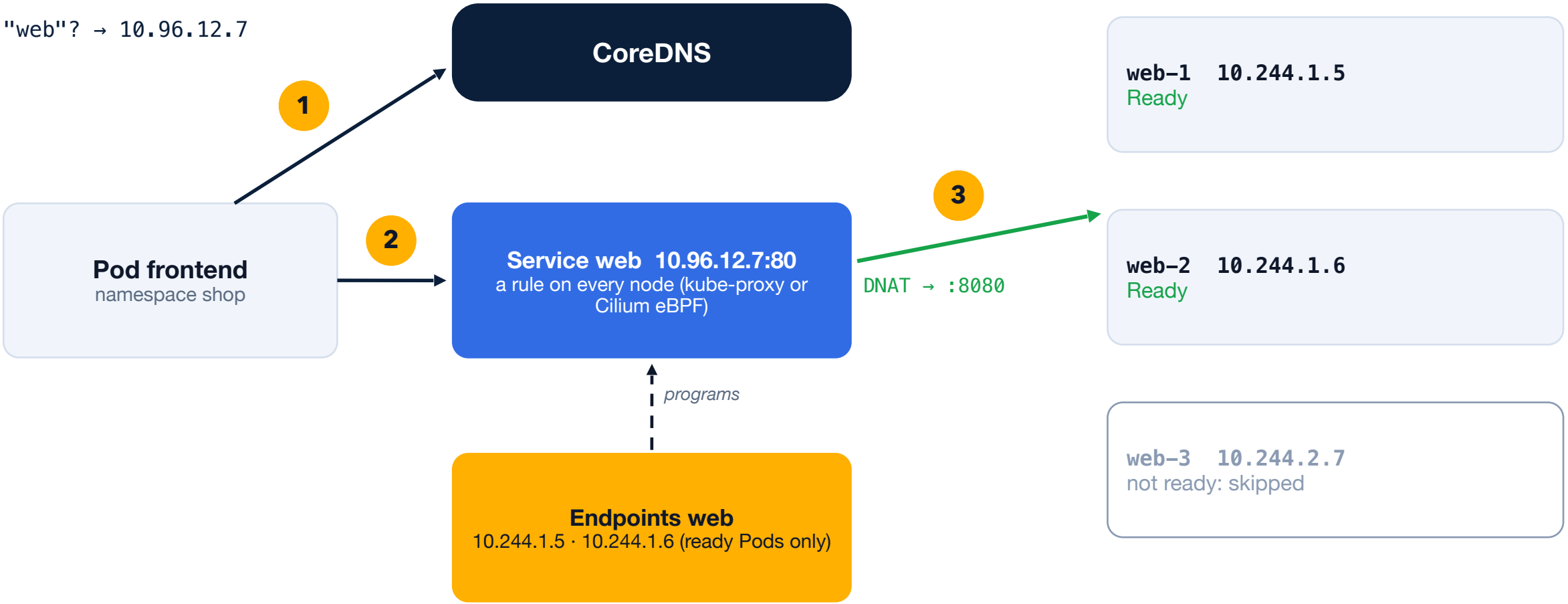
→ web in billing

```
web.billing.svc.cluster.local
```

→ fully qualified

Configure apps with Service names, never Pod IPs.

Following a request inside the cluster



No proxy in the path: the Service IP is rewritten to a Pod IP on the client's node, then the CNI delivers it.

Headless Services: DNS without a virtual IP

ClusterIP Service

```
$ nslookup web
Name:    web.shop.svc.cluster.local
Address: 10.96.12.7     # one virtual IP
```

One stable IP. A rule on the node picks a Pod for every connection.

Headless Service · clusterIP: None

```
$ nslookup db
Name:    db.shop.svc.cluster.local
Address: 10.244.1.5
Address: 10.244.1.6     # every ready Pod
$ nslookup db-0.db
Address: 10.244.1.5     # one Pod, by name
```

No virtual IP, no NAT: the client gets Pod IPs and connects directly.

```
apiVersion: v1
kind: Service
metadata: { name: db }
spec:
  clusterIP: None    # headless
  selector: { app: db }
  ports: [{ port: 5432 }]
```



StatefulSets

Stable per-Pod names: db-0.db, db-1.db



Peer discovery

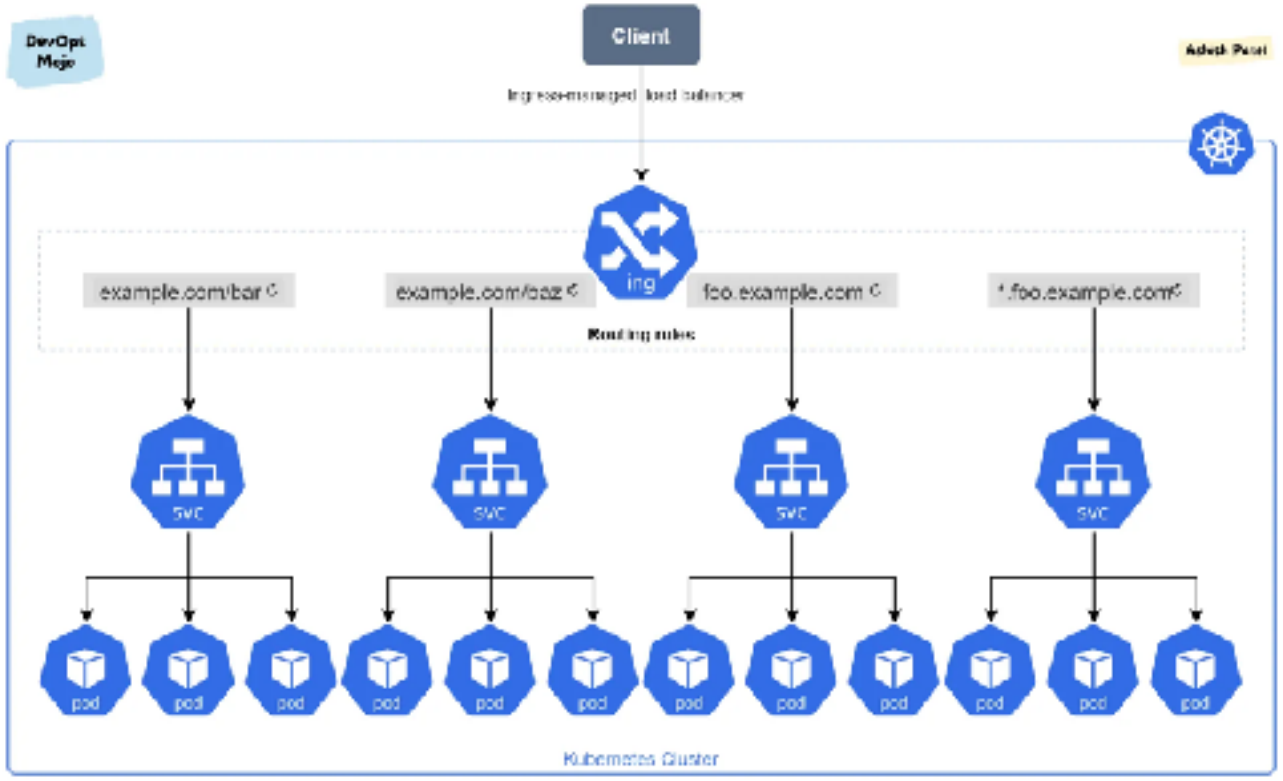
Kafka, Cassandra, etcd find each other



Client-side balancing

gRPC spreads its long-lived connections itself

Ingress: HTTP routing into the cluster



- Layer 7 (HTTP/HTTPS) routing to Services
- **Host-based:** foo.example.com → svc A
- **Path-based:** /api → svc B, / → svc C
- **TLS termination,** often with cert-manager + Let's Encrypt
- Needs an Ingress controller to do anything

Ingress in YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
spec:
  ingressClassName: traefik
  tls:
  - hosts: ["shop.example.com"]
    secretName: shop-tls
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: api, port: { number: 80 } }
      - path: /
        pathType: Prefix
        backend:
          service: { name: web, port: { number: 80 } }
```



Heads-up for 2026

The community ingress-nginx controller is retired (maintenance ended March 2026).

Existing Ingress objects still work with other controllers. For new work, prefer the Gateway API.

BREAK

Break & cluster setup

Grab a coffee, then get a local cluster running with the install guide.

github.com/saman2000hoseini/k8s-workshop

Your cluster for today: minikube

One command

minikube start gives you a real Kubernetes cluster in a container.

Addons built in

metrics-server, ingress, dashboard: one enable away.

NetworkPolicy ready

Start with --cni=cilium so Exercise 4 works.

Disposable

Broke it? minikube delete, minikube start, and you're back.

terminal

```
# start the workshop cluster
$ minikube start --cni=cilium

# is it up?
$ minikube status
$ kubectl get nodes

# addons and web UI
$ minikube addons list
$ minikube dashboard

# done for the day
$ minikube stop      # pause, keep state
$ minikube delete    # remove everything
```

Installation guide: minikube

macOS

```
# Homebrew
# (kubectl comes with it)
$ brew install minikube
```

Windows

```
# PowerShell
$ winget install Kubernetes.minikube
$ winget install Kubernetes.kubectl
```

Linux (x86_64)

```
$ curl -LO https://github.com/\
kubernetes/minikube/releases/\
latest/download/minikube-linux-amd64
$ sudo install minikube-linux-amd64 \
    /usr/local/bin/minikube
$ alias kubectl="minikube kubectl --"
```

Then, on every OS

```
$ minikube start --cni=cilium            # first start downloads ~1.5 GB
$ kubectl get nodes                      # minikube   Ready   control-plane
$ git clone https://github.com/saman2000hoseini/k8s-workshop
$ cd k8s-workshop && ./00-setup/verify.sh
```

Needs: Docker Desktop, OrbStack or Podman running, ~4 GB RAM. **Full guide:** `k8s-workshop/00-setup/README.md`

Setup checklist

1. Clone github.com/saman2000hoseini/k8s-workshop
2. Docker, OrbStack or Podman is running
3. minikube and kubectl are installed
4. minikube start --cni=cilium; the node is Ready
5. ./00-setup/verify.sh says All good. Done? Help a neighbour

```
$ minikube start --cni=cilium
$ kubectl get nodes
NAME       STATUS   ROLES           AGE
minikube   Ready    control-plane   1m

$ kubectl run hello --image=nginx:1.27 \
    --restart=Never
$ kubectl get pod hello          # Running?
$ kubectl delete pod hello
```

PART 2

Hands-on

Deploy, break and heal, configure, expose and debug a real app on your own cluster.



kubectl essentials

get

`kubectl get pods -o wide`

List resources (-A all namespaces, -w watch)

describe

`kubectl describe pod web-7d9`

Details + Events: start here when debugging

apply

`kubectl apply -f k8s/`

Create/update from YAML (declarative)

logs

`kubectl logs deploy/web -f --previous`

Container logs; --previous = last crash

exec

`kubectl exec -it web-7d9 -- sh`

Shell into a running container

port-forward

`kubectl port-forward svc/web 8080:80`

Reach a Service from your laptop

debug

`kubectl debug -it web-7d9 --image=busybox`

Attach a temporary debug container

delete

`kubectl delete -f k8s/`

Remove what you applied

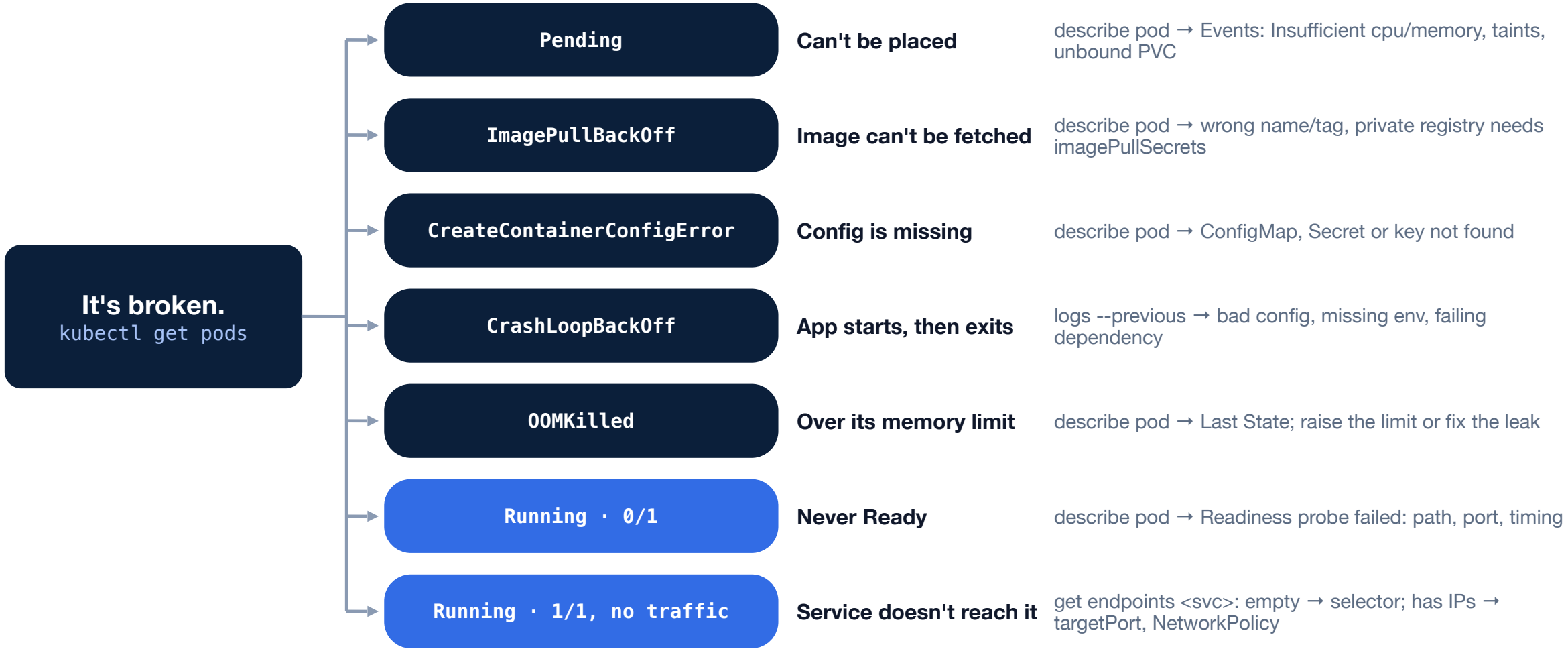
```
$ kubectl create deployment web --image=nginx:1.27 --dry-run=client -o yaml > web.yaml
```

Debugging playbook

Symptom	Likely cause	First command
Pending	No node fits: requests too big, taints, unbound PVC	kubectl describe pod <pod> → Events
ImagePullBackOff	Typo in image/tag, private registry without imagePullSecret	kubectl describe pod <pod>
CrashLoopBackOff	App exits on start: bad config, missing env, failing dependency	kubectl logs <pod> --previous
OOMKilled	Memory limit too low, or a leak	kubectl describe pod → Last State
Running, no traffic	Selector mismatch, readiness failing, wrong targetPort	kubectl get endpoints <svc>
Works locally, not in cluster	DNS name, NetworkPolicy, missing Secret/ConfigMap	kubectl debug -it <pod> --image=busybox

```
$ kubectl get events -n shop --sort-by=.lastTimestamp
```

Debugging decision tree



`Still stuck?  kubectl get events --sort-by=.lastTimestamp  ·  kubectl debug -it <pod> --image=busybox`

HANDS-ON

Exercise 1: deploy, break, heal, roll back

1. Create a Deployment of nginx:1.27 with 3 replicas
2. Delete one Pod and watch the ReplicaSet replace it
3. Scale to 5 replicas
4. Update to a broken tag (nginx:nope). What status do you see? Does the old version keep serving?
5. Roll back and check the rollout history

Files `k8s-workshop/01-deploy-break-heal`

```
$ kubectl create deployment web \
    --image=nginx:1.27 --replicas=3
$ kubectl get pods -l app=web -w

$ kubectl delete pod <one-of-the-pods>

$ kubectl scale deploy/web --replicas=5

$ kubectl set image deploy/web nginx=nginx:nope
$ kubectl rollout status deploy/web
$ kubectl get pods -l app=web

$ kubectl rollout undo deploy/web
$ kubectl rollout history deploy/web
```

HANDS-ON

Exercise 2: config & state

1. Create a ConfigMap and a Secret, and inject them into your Deployment as env vars
2. Exec into a Pod and print the variables. Decode the Secret with kubectl. What does that tell you?
3. Change the ConfigMap. Did the Pod notice? Fix it with a rollout restart
4. Follow the official StatefulSet tutorial: scale it, delete a Pod, check its name and data

```
$ kubectl create configmap web-config \
    --from-literal=LOG_LEVEL=debug
$ kubectl create secret generic db-creds \
    --from-literal=DB_PASSWORD=s3cr3t
$ kubectl set env deploy/web \
    --from=configmap/web-config
$ kubectl set env deploy/web \
    --from=secret/db-creds

$ kubectl exec deploy/web -- env | grep DB
$ kubectl get secret db-creds \
    -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

Files `k8s-workshop/02-config-and-state`

https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/

HANDS-ON

Exercise 3: expose and discover

1. Expose your Deployment with a ClusterIP Service
2. From a throwaway Pod, reach it by DNS name. Then try <svc>.<ns>.svc.cluster.local
3. Reach it from your laptop with port-forward
4. Break it: point the Service selector at app=nope. What happens to the Endpoints? Then fix it

```
$ kubectl expose deploy/web --port=80
$ kubectl get svc,endpoints

$ kubectl run tmp --rm -it --image=busybox \
    --restart=Never -- wget -qO- http://web

$ kubectl port-forward svc/web 8080:80
# open http://localhost:8080

$ kubectl patch svc web -p \
    '{"spec":{"selector":{"app":"nope"}}}'
$ kubectl get endpoints web
```

Files `k8s-workshop/03-expose-and-discover`

TIMED CHALLENGE

Challenge round 1: fix it fast

1. Apply broken.yaml
2. Debug: get, describe, logs
3. Fix it
4. `./check.sh` says PASS

c1 · Stuck in Pending

`challenges/c1-pending`

Symptom two api Pods never start.

Goal both Running, with a sensible CPU request.

c2 · Missing greeting

`challenges/c2-config`

Symptom greeter never starts.

Goal it prints the greeting from its ConfigMap.

c3 · Connection refused

`challenges/c3-service`

Symptom healthy Pods, but the shop Service refuses connections.

Goal http://shop.c3 returns the nginx page.

c4 · Never Ready

`challenges/c4-readiness`

Symptom Pods Running but 0/1 Ready; the Service has no endpoints.

Goal both Ready, keeping the readiness probe.

Files `k8s-workshop/challenges` · Stuck? Each README has a hint. Answers when time is up.

Round 1 debrief: what was wrong?

Challenge	What was wrong	How to spot it	Fix
c1 · Pending	cpu: "64" requested: no node has 64 CPUs	describe pod → Events: Insufficient cpu	A realistic request, e.g. 100m
c2 · Won't start	Env uses key greeting; the ConfigMap key is GREETING	STATUS CreateContainerConfigError; describe pod → couldn't find key	Fix the key name (case-sensitive)
c3 · Connection refused	targetPort 8080, but nginx listens on 80	Endpoints list Pod IPs, so it's not the selector	targetPort: 80
c4 · 0/1 Ready	Readiness probe checks /healthz: nginx returns 404	describe pod → Readiness probe failed: 404	Probe a path that exists (/)

The pattern: get → describe (read the Events) → compare what you asked for with what exists.

BREAK

Short break

Next: Part 3 · Advanced topics. Keep your cluster running: the advanced labs use it.



PART 3

Advanced topics

Advanced networking, then running workloads in production: resources, autoscaling, RBAC, security, and static Pods.

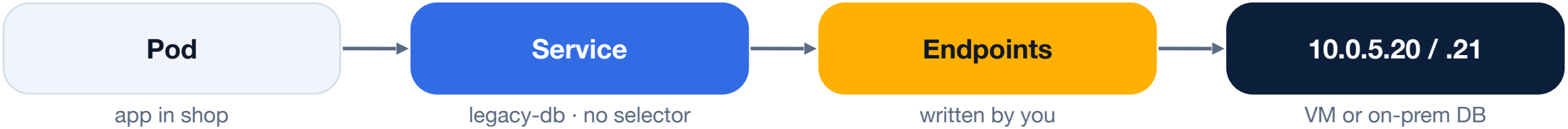


06

Advanced Networking

Custom endpoints, Gateway API and NetworkPolicy

Custom endpoints: Services without a selector



service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: legacy-db
  namespace: shop
spec:                  # no selector!
  ports:
  - name: pg
    port: 5432
```

endpoints.yaml

```yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: legacy-db           # = Service name: that's the link
  namespace: shop
subsets:
- addresses:
  - ip: 10.0.5.20
  - ip: 10.0.5.21
  ports:
  - name: pg                # = Service port name
    port: 5432
```

Reaching things outside the cluster

	ExternalName	Custom endpoints
Target	DNS hostname	IP addresses you list
How	DNS CNAME	ClusterIP + kube-proxy
Port mapping	No	Yes
Load balancing	No	Across your IPs
Health checks	No	No: you maintain the list
Use for	Managed DB / SaaS API	VMs, on-prem, migrations

externalname.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ExternalName
  externalName: pg.prod.example.com
```

Same name for clients when the dependency moves into the cluster.

```
$ kubectl get endpoints legacy-db
$ kubectl run tmp --rm -it --image=busybox --restart=Never -- nc -zv legacy-db 5432
```

From Endpoints to EndpointSlices

Endpoints · v1, deprecated since 1.33

```
endpoints/web
10.244.0.4  10.244.0.7  10.244.1.2  10.244.1.9
10.244.2.3  10.244.2.8  …  every Pod of the Service
in ONE object
```

- One Pod changes → the whole object is rewritten and sent to every node
- Capped at 1000 addresses; bigger Services are truncated

EndpointSlices · discovery.k8s.io/v1

web-x7k	web-p2m	web-q9d
≤ 100 Pods	≤ 100 Pods **only this slice changes**	≤ 100 Pods

- One Pod changes → only its slice is updated
- Per endpoint: ready / serving / terminating, node and zone, IPv4 and IPv6

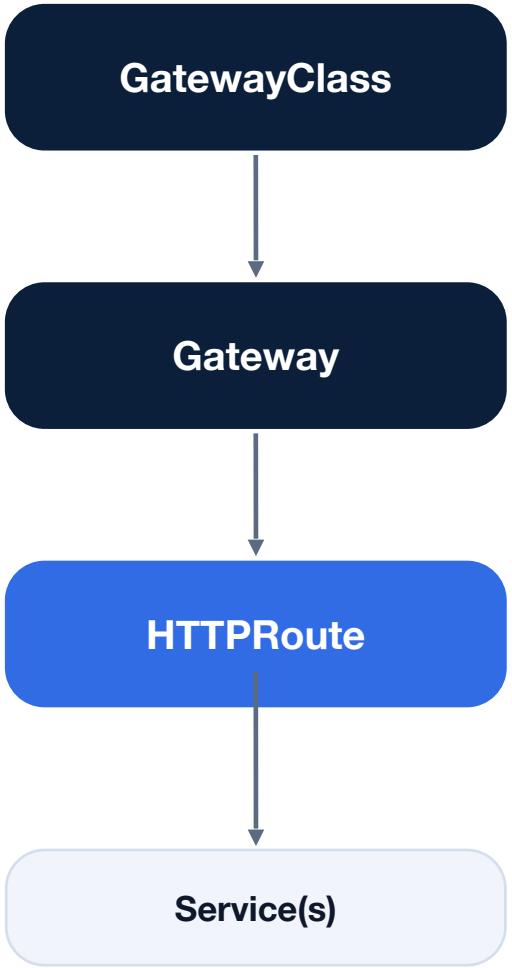
Migrate

Read: `kubectl get endpoints web` → `kubectl get endpointslices -l kubernetes.io/service-name=web`

Write: hand-written Endpoints → EndpointSlice with labels service-name + managed-by

Already today: kube-proxy and Cilium read only slices; your Endpoints are mirrored into them

Gateway API: the next generation



Infrastructure provider

which controller implements it

Platform / cluster operator

listeners, ports, TLS certificates

Application developer

hosts, paths, headers → Services

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop
spec:
  parentRefs:
  - name: public-gw          # owned by platform
  hostnames: ["shop.example.com"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: /api }
    backendRefs:
    - name: api-v1
      port: 80
      weight: 90             # canary: 90/10
    - name: api-v2
      port: 80
      weight: 10
```

NetworkPolicy: a firewall for Pods

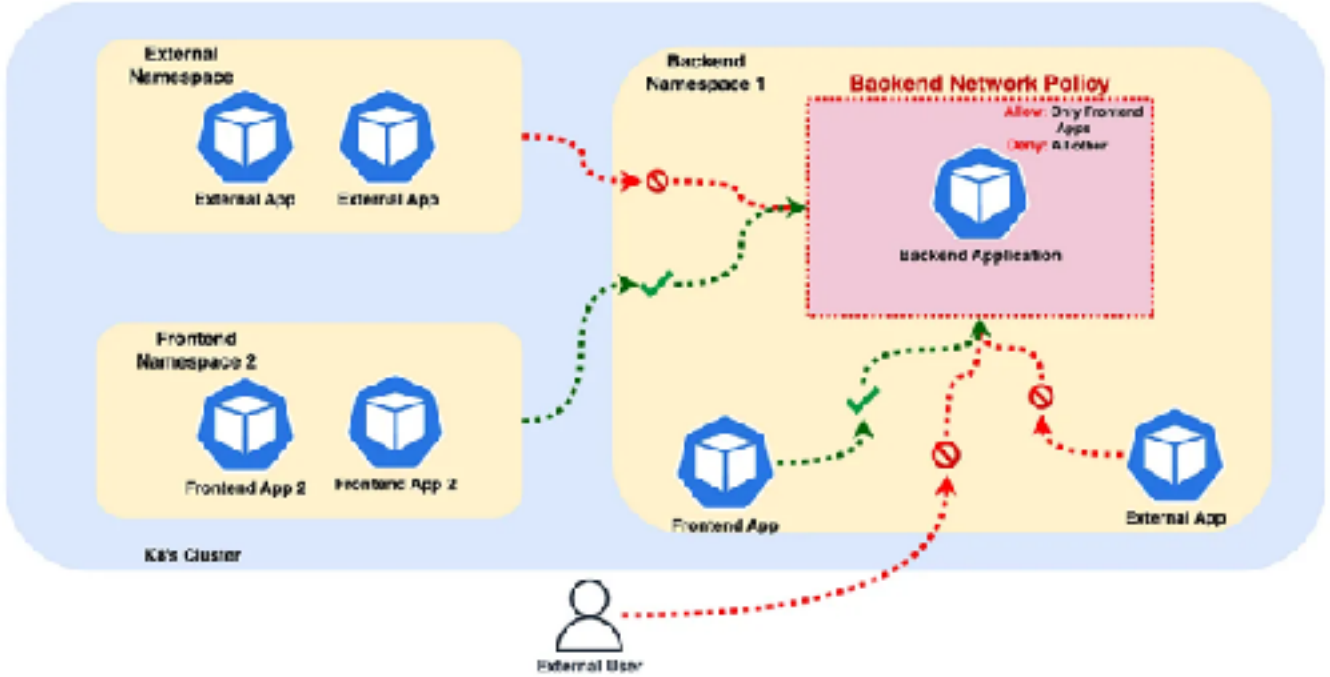


Diagram: Anvesh Muppeda

- Default: all Pods accept traffic from everywhere
- Selects Pods by label; allows ingress / egress
- Selected Pods deny anything not allowed
- Enforced by the CNI (Calico, Cilium…): no support = no effect

Start with default-deny per namespace, then open exactly the paths you need: frontend → api, api → db, everyone → DNS.

NetworkPolicy in YAML

```yaml
# 1. lock the namespace down
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}          # all Pods
  policyTypes: ["Ingress"]
```

Policies only add allows. Denying egress? Allow DNS (port 53) too.

```yaml
# 2. allow web -> api on 8080
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-from-web
  namespace: shop
spec:
  podSelector:
    matchLabels: { app: api }
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: web }
    ports:
    - port: 8080
```

HANDS-ON

Exercise 4: lock it down

1. Apply a default-deny ingress policy in your namespace
2. Try to reach web from a throwaway Pod. It should time out
3. Allow only Pods labelled access=true to reach app=web. Test with and without the label
4. Challenge: a selector-less Service + Endpoints for an outside IP. Reach it by name

Files `k8s-workshop/04-network-policy`

```
$ kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny }
spec:
  podSelector: {}
  policyTypes: [Ingress]
EOF

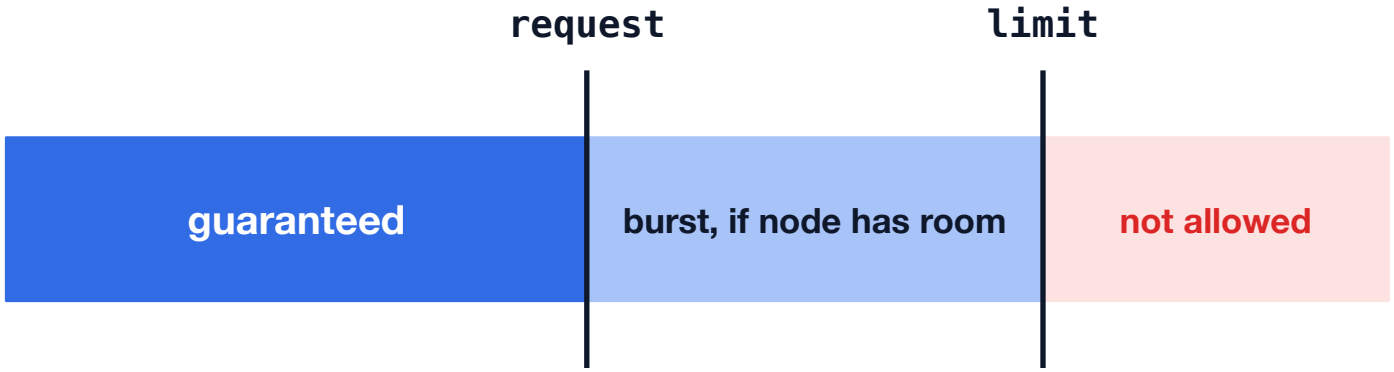
$ kubectl run tmp --rm -it --image=busybox \
    --restart=Never -- wget -T 3 -qO- http://web
$ kubectl run tmp --rm -it --image=busybox \
    --labels=access=true --restart=Never \
    -- wget -T 3 -qO- http://web
```

07

Running in Production

Resources, scheduling, autoscaling, observability,
RBAC and security

Requests & limits



Request	Used by the scheduler to place the Pod. Reserved for you on the node.
CPU over limit	Throttled: slower, never killed.
Memory over limit	OOMKilled: the container restarts.
No requests at all	Scheduler is blind; first to be evicted under pressure.

```yaml
resources:
  requests:
    cpu: 250m          # 0.25 core
    memory: 256Mi
  limits:
    memory: 256Mi      # = request
    # cpu limit: optional, see notes
```

CPU

1 = one vCPU/core
500m = half a core
Same on any node size

Memory

Mi, Gi = powers of 2
M, G = powers of 10
256Mi ≠ 256M

Quality of Service classes

Guaranteed

requests = limits for CPU and memory, on every container

Evicted last

Burstable

at least one request or limit set, but not equal

Evicted when above its request

BestEffort

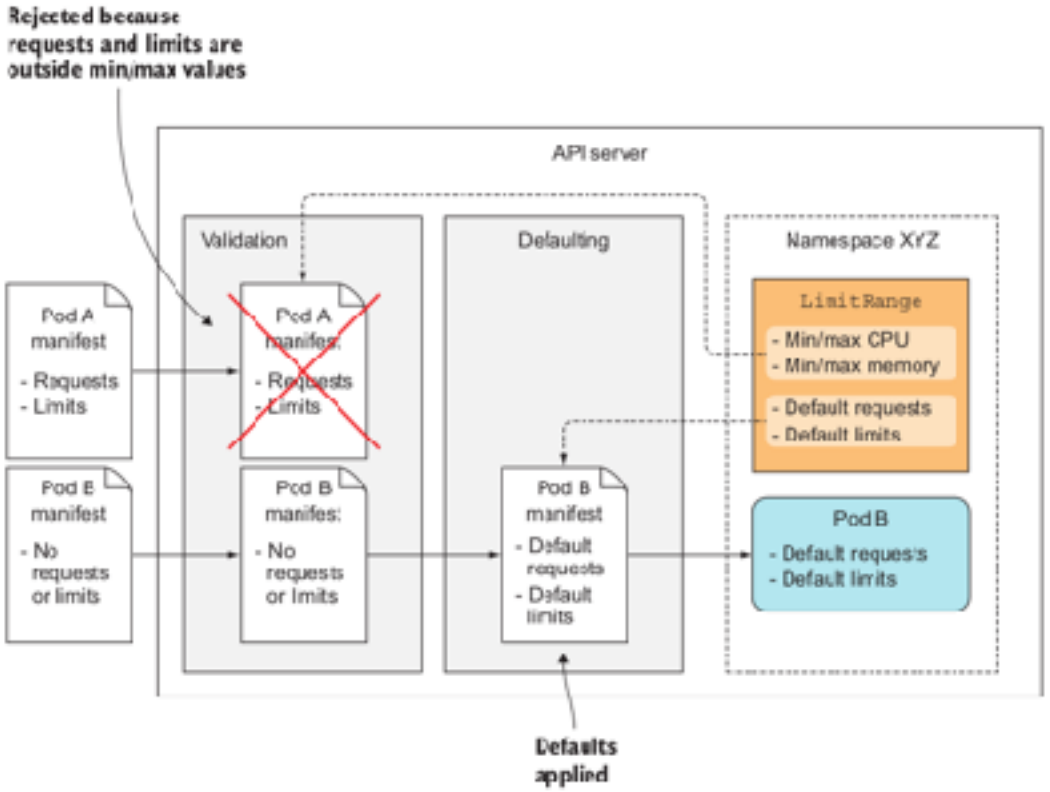
no requests or limits at all

Evicted first



eviction order under node memory pressure

Sharing a cluster fairly



ResourceQuota: total per namespace
LimitRange: defaults + min/max per container

```yaml
apiVersion: v1
kind: ResourceQuota
metadata: { name: team-a, namespace: team-a }
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 64Gi
    pods: "100"
```

```yaml
apiVersion: v1
kind: LimitRange
metadata: { name: defaults, namespace: team-a }
spec:
  limits:
  - type: Container
    defaultRequest: { cpu: 100m, memory: 128Mi }
    default:        { memory: 256Mi }
    max:            { cpu: "2", memory: 2Gi }
```

Controlling where Pods run

nodeSelector

Only nodes with these labels

```
disktype: ssd
```

Node affinity

Required (hard) or preferred (weighted) rules on node labels

```
zone In [a, b]
```

Pod (anti-)affinity

Run near, or away from, other Pods

```
not with another app=web
```

Topology spread

Spread replicas evenly across zones / nodes

```
maxSkew: 1 per zone
```

Taints & tolerations

Nodes repel Pods unless the Pod tolerates the taint

```
gpu=true:NoSchedule
```

nodeName

Bypass the scheduler entirely. Avoid it

```
node-3
```

Autoscaling at three levels



HPA

Horizontal Pod Autoscaler

More or fewer replicas based on CPU, memory or custom metrics

Best for: stateless services, the common case



VPA

Vertical Pod Autoscaler

Right-sizes requests per container; recommend-only mode is a safe start

Best for: finding the right requests



Nodes

Cluster Autoscaler / Karpenter

Adds nodes when Pods are Pending, removes idle ones

Best for: cloud cost & capacity

Event-driven? KEDA scales on queue length, Kafka lag, cron windows… and can scale to zero.

HorizontalPodAutoscaler

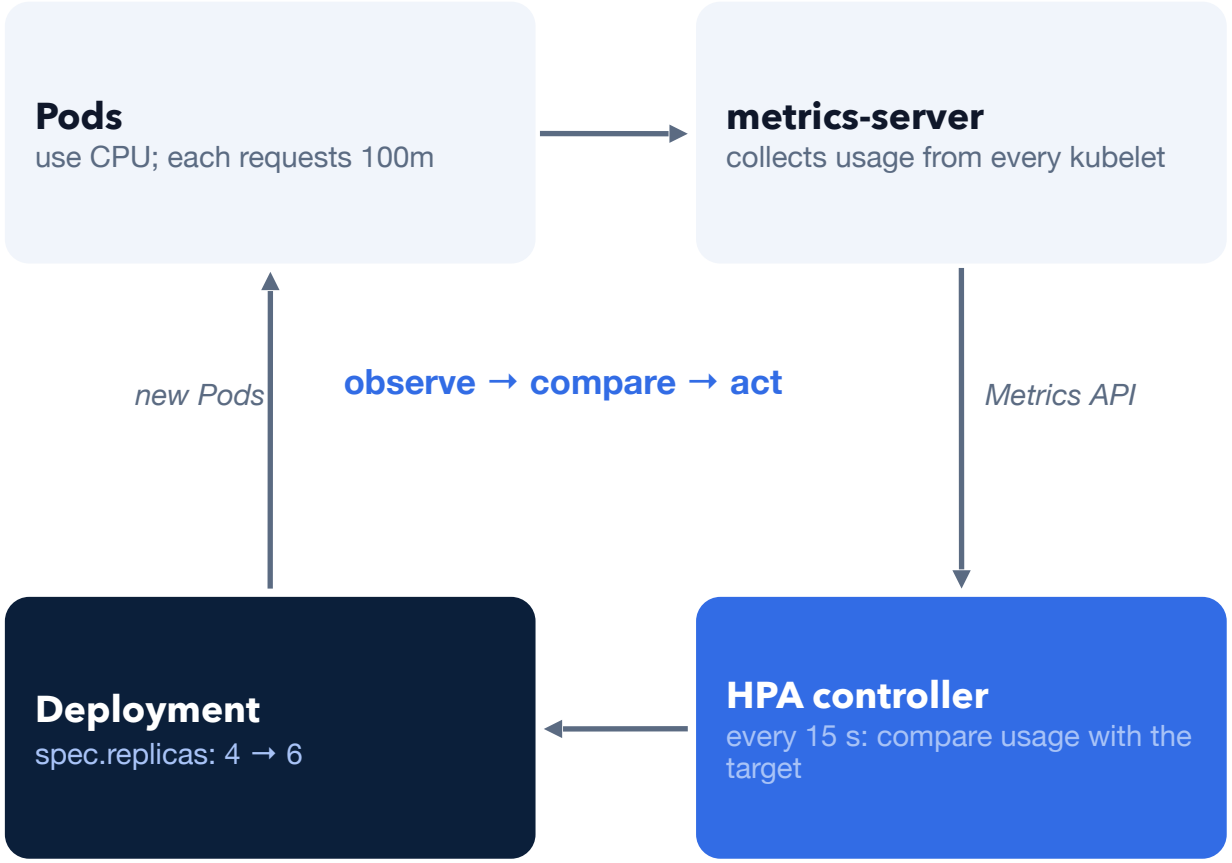
```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # % of the request
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Checklist

- ✓ metrics-server is installed (kubectl top pods works)
- ✓ The Deployment sets CPU requests: utilization is % of request
- ✓ minReplicas ≥ 2 for availability
- ✓ Don't hard-code replicas in the Deployment if GitOps applies it: it fights the HPA

```
$ kubectl get hpa web -w
```

How the HPA decides



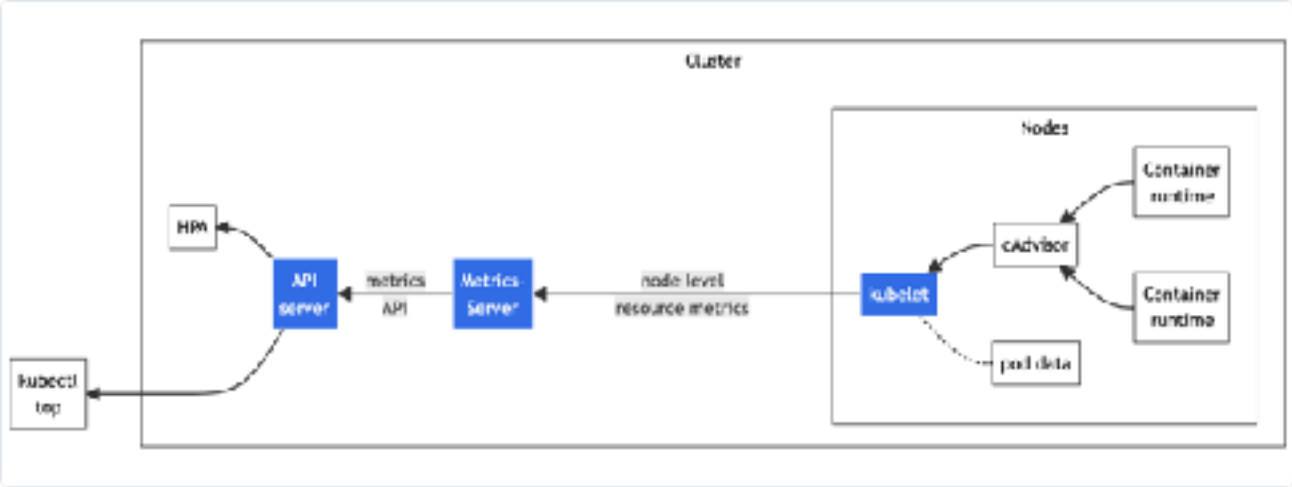
The math

`desired = ceil(current × usage / target)`

```
4 Pods at 90% CPU, target 60%
ceil(4 × 90 / 60) = 6 Pods
```

- Usage is % of the request: no request, no autoscaling
- Within ±10% of the target: no change
- Scale up fast; scale down only after 5 min of lower usage
- Always between minReplicas and maxReplicas

Observability: metrics, logs, traces



Metrics

metrics-server for kubectl top + HPA. Prometheus + Grafana for monitoring and alerts

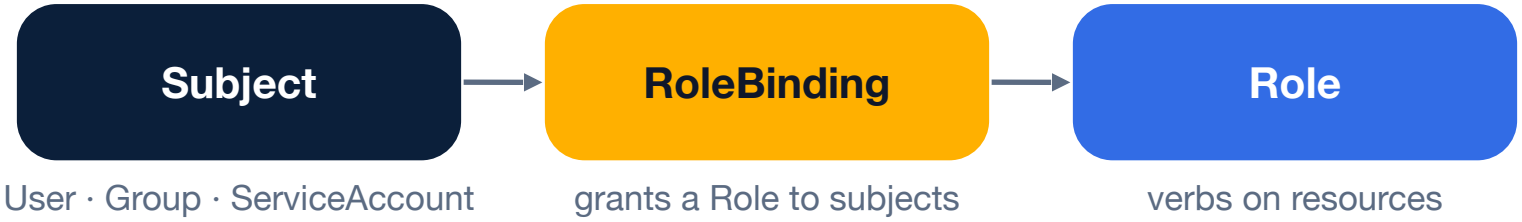
Logs

Log to stdout/stderr. A DaemonSet (Fluent Bit, Vector) ships them to Loki or Elasticsearch

Traces

Instrument with OpenTelemetry; view in Jaeger, Tempo or your APM

RBAC: who can do what, where



Scope	Permissions	Grant
Namespace	Role	RoleBinding
Cluster-wide	ClusterRole	ClusterRoleBinding

```
$ kubectl auth can-i list pods -n shop
$ kubectl auth can-i create deploy -n shop \
    --as=system:serviceaccount:shop:ci
```

Deny by default

Rules only add permissions. There is no "deny" rule.

Users aren't objects

Humans come from certs or OIDC. ServiceAccounts are objects.

Verbs

get · list · watch · create · update · patch · delete

Start from built-ins

view · edit · admin. Never cluster-admin for apps.

Role & RoleBinding in YAML

role.yaml · what is allowed

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: shop          # only in shop
rules:
- apiGroups: [""]          # "" = core group
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list"]
```

rolebinding.yaml · who gets it

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-reads-pods
  namespace: shop
subjects:                  # who
- kind: ServiceAccount
  name: ci
  namespace: shop
- kind: Group
  name: dev-team           # from OIDC
roleRef:                   # what (immutable)
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Reuse built-ins: a RoleBinding can point at a ClusterRole (view, edit, admin) and grant it in one namespace only.

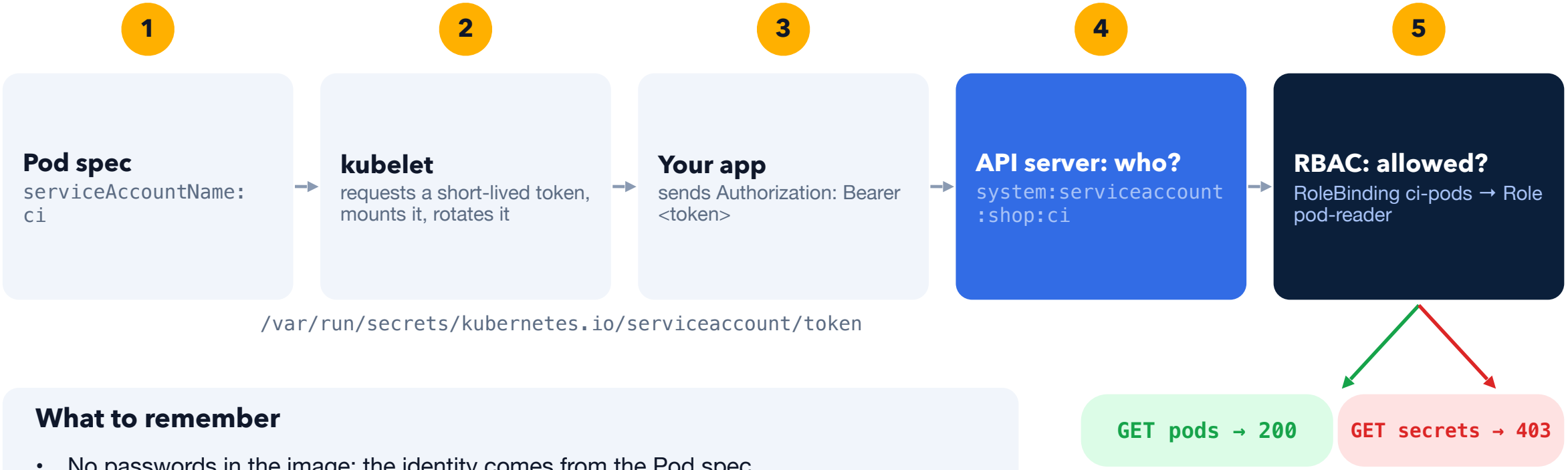
ServiceAccounts: identity for Pods

- **ServiceAccounts** are API objects for workloads; users are humans (certs, OIDC)
- **default** SA in every namespace, used if you set none
- Token mounted at /var/run/secrets/kubernetes.io/serviceaccount: short-lived, auto-rotated
- One SA per app, bound to a least-privilege Role
- App never calls the API? Disable the token mount
- Cloud: workload identity maps SA → IAM role. No static keys

```
$ kubectl create serviceaccount ci -n shop
$ kubectl create token ci -n shop --duration=1h
$ kubectl auth whoami
```

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dashboard
  namespace: shop
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dashboard
  namespace: shop
spec:
  template:
    spec:
      serviceAccountName: dashboard     # identity
      automountServiceAccountToken: true
      containers:
      - name: app
        image: shop/dashboard:1.4
```

How a Pod talks to the API



What to remember

- No passwords in the image: the identity comes from the Pod spec
- Tokens expire and rotate; client libraries reload them automatically
- Same RBAC rules as for humans, so check with kubectl auth can-i --as
- App never calls the API? automountServiceAccountToken: false

Securing your workloads

```yaml
spec:
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    seccompProfile: { type: RuntimeDefault }
  containers:
  - name: api
    image: ghcr.io/acme/api@sha256:4f3c…
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

```
$ kubectl label ns shop \
    pod-security.kubernetes.io/enforce=restricted
```

Least privilege

Non-root, read-only FS, drop all capabilities

Scan & pin images

Trivy / Grype in CI. Pin by digest. Minimal or distroless bases

Pod Security Admission

Enforce baseline or restricted per namespace

Secrets & RBAC

No cluster-admin for apps. Disable unused SA tokens

HANDS-ON

Exercise 5: right-size & autoscale

1. Add CPU/memory requests and limits to your Deployment. Check its QoS class
2. Install metrics-server with ./install-metrics-server.sh and run kubectl top pods
3. Create an HPA at 50% CPU and generate load. Watch replicas grow, then shrink
4. Challenge: give a container a 32Mi memory limit and make it allocate more. What happens?

```
$ kubectl set resources deploy/web \
    --requests=cpu=100m,memory=64Mi \
    --limits=memory=128Mi
$ kubectl get pod -l app=web \
    -o jsonpath='{.items[0].status.qosClass}'

$ ./install-metrics-server.sh
$ kubectl top pods

$ kubectl autoscale deploy/web \
    --cpu-percent=50 --min=1 --max=10
$ kubectl apply -f load-generator.yaml
$ kubectl get hpa,pods -w
```

Files `k8s-workshop/05-resources-and-autoscaling`

https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/
https://labs.iximiuz.com/challenges/start-pod-with-limited-resources

HANDS-ON

Exercise 6: least privilege with RBAC

1. Create a ServiceAccount ci in namespace shop
2. Create a Role that can only read Pods, and bind it to ci
3. Use kubectl auth can-i --as to check what ci can and cannot do
4. Challenge: let ci read logs too, or bind the built-in view ClusterRole instead

```
$ kubectl create ns shop
$ kubectl create sa ci -n shop

$ kubectl create role pod-reader -n shop \
    --verb=get,list,watch --resource=pods
$ kubectl create rolebinding ci-pods -n shop \
    --role=pod-reader --serviceaccount=shop:ci

$ kubectl auth can-i list pods -n shop \
    --as=system:serviceaccount:shop:ci   # yes
$ kubectl auth can-i delete pods -n shop \
    --as=system:serviceaccount:shop:ci   # no
$ kubectl auth can-i --list -n shop \
    --as=system:serviceaccount:shop:ci
```

Files `k8s-workshop/06-rbac`

https://kubernetes.io/docs/reference/access-authn-authz/rbac/
https://kubernetes.io/docs/concepts/security/service-accounts/

TIMED CHALLENGE

Challenge round 2: fix it fast

1. Apply broken.yaml
2. Debug: get, describe, logs
3. Fix it
4. `./check.sh` says PASS

c5 · Locked out

`challenges/c5-network-policy`

Symptom frontend can't reach api, despite an allow policy.

Goal frontend reaches api; any other Pod is still blocked.

c6 · Forbidden

`challenges/c6-rbac`

Symptom CI runs as deployer and every deploy is forbidden. Two mistakes.

Goal deployer manages Deployments in c6, and nothing more.

c7 · Blind autoscaler

`challenges/c7-autoscaling`

Symptom the HPA's TARGETS column says <unknown>.

Goal the HPA shows a real CPU percentage (needs metrics-server).

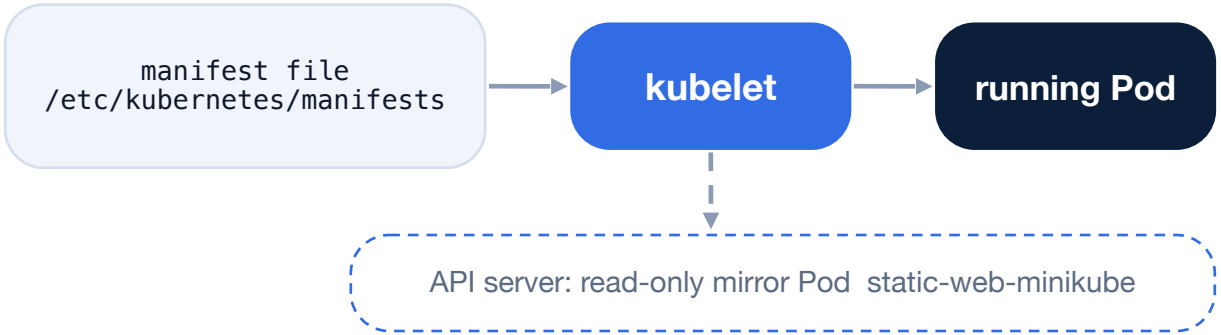
Files `k8s-workshop/challenges` · Stuck? Each README has a hint. Answers when time is up.

Round 2 debrief: what was wrong?

Challenge	What was wrong	How to spot it	Fix
c5 · Blocked	Allow policy selects role=frontend; Pods are labelled app=frontend	get pods --show-labels vs the policy's podSelector	Match the label
c6 · Forbidden	Role grants deployments in group "" (they're in apps) and binds deploy-bot	auth can-i --as=…:deployer; kubectl api-resources	apiGroups: [apps]; subject deployer
c7 · <unknown>	No CPU request: utilization is a % of the request	describe hpa (often vague); no resources in the Pod spec	Add requests.cpu, wait a minute

Selectors, subjects and requests: small mismatches, total failure, and the error rarely names the cause.

Static Pods: how a node bootstraps itself



- **No API server needed:** the kubelet runs manifests straight from disk
- **Mirror Pod:** visible in kubectl, but read-only; kubectl delete just brings it back
- **Manage the file:** edit it to update, delete it to stop
- **Node setup:** kubeadm writes etcd, apiserver, scheduler and controller-manager here; upgrades rewrite them
- **Limits:** one node, no ConfigMaps, Secrets or ServiceAccounts. For every node, use a DaemonSet

```
demo on minikube

# what the kubelet runs from disk
$ minikube ssh -- ls /etc/kubernetes/manifests
etcd.yaml  kube-apiserver.yaml
kube-controller-manager.yaml  kube-scheduler.yaml

# drop a manifest in: the kubelet starts it
$ minikube cp templates/static-pod.yaml \
    /etc/kubernetes/manifests/static-web.yaml
$ kubectl get pods
static-web-minikube   1/1   Running

# kubectl delete? it comes right back
# remove the file to stop it
$ minikube ssh -- sudo rm \
    /etc/kubernetes/manifests/static-web.yaml
```

Where to go next

Helm

Package manager: charts with values per environment

Kustomize

Template-free overlays, built into kubectl -k

GitOps

Argo CD / Flux keep the cluster synced with Git

Operators & CRDs

Postgres, Kafka, cert-manager… run by controllers

Gateway API & mesh

Advanced traffic: canaries, mTLS, retries

Policy engines

Kyverno / OPA Gatekeeper enforce rules at admission

Keep practicing

iximiuz Labs	labs.iximiuz.com	Hands-on challenges and playgrounds
Killercoda	killercoda.com	Free in-browser Kubernetes scenarios
SadServers	sadservers.com	Fix-the-broken-system puzzles
Kubernetes docs	kubernetes.io/docs/tutorials	Official tutorials and concepts
CKAD / CKA	training.linuxfoundation.org	Developer and admin certifications

Key takeaways

1. Kubernetes is control loops reconciling desired state
2. Deployments for stateless apps; StatefulSets only when you need identity
3. Readiness and liveness probes are part of your app's contract
4. Talk to Services by DNS name; restrict traffic with NetworkPolicies
5. Always set requests; then autoscale, secure, and debug with describe → logs → debug

THANK YOU

Questions?

Kubernetes Fundamentals · Linux Fest 2026

Saman Hoseini

